

From Characters to Transformers

Next-token prediction, attention, and a decoder-only language model

Prof. Nipun Batra

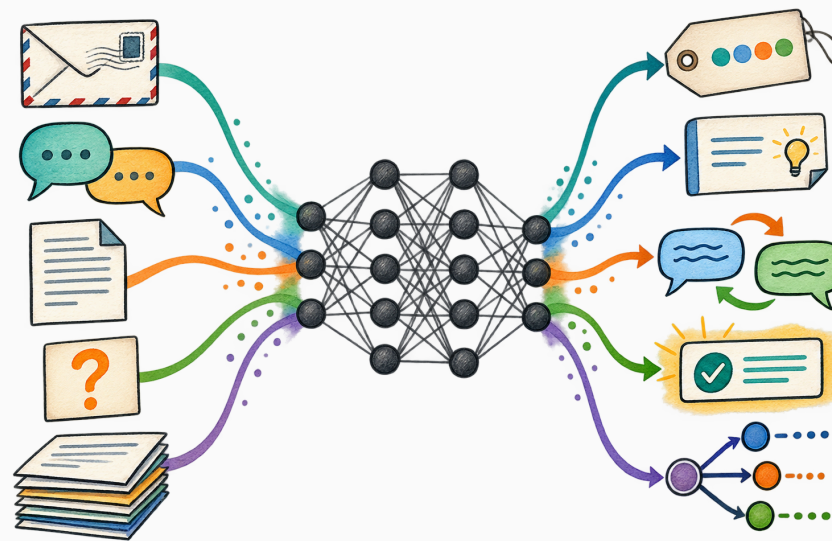
ES 667 · Deep Learning · IIT Gandhinagar

Lecture 1 · Next-token prediction

How can a neural network work with **language**?

CLASSIFY
UNDERSTAND
COMPRESS
TRANSLATE
ANSWER
GENERATE

email → spam?
review → sentiment
article → summary
English → Hindi
context + question → answer
prompt → continuation



Examples: MIT 6.S191 L2 and Stanford CME 295 L1.

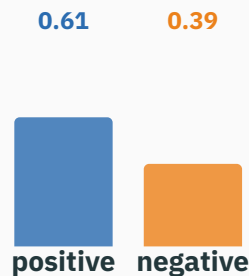
One input → one label

REVIEW

“The acting was wonderful, but the ending was disappointing.”

TEXT INPUT

x



We have seen classification before. What changes when the input is text?

Intent classification

UTTERANCE

Book a flight to Delhi

UTTERANCE

What is my account balance?

UTTERANCE

Cancel my booking

LABEL



BOOK_FLIGHT

LABEL



CHECK_BALANCE

LABEL



CANCEL_BOOKING

$x \rightarrow y$: a text input of any length, one class label

Token-level prediction: named entities

Sundar	Pichai	visited	IIT	Gandhinagar	on	Monday.
PERSON	PERSON	O	ORG	ORG	O	DATE

INPUT

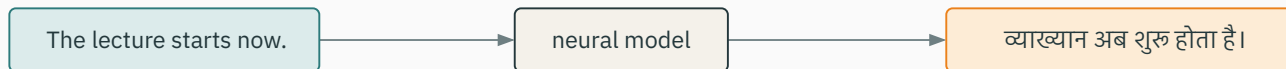
OUTPUT

a sequence of T tokens

one class label for each of the T tokens

One token in, one label out.

Translation changes the sequence length



ENGLISH

HINDI

4 displayed words

5 displayed words

One input word does not always correspond to one output word.

Summarize a notice

IITGN NOTICE



The library will remain open until midnight during examinations.
Extended hours begin Monday.
Entry requires a valid institute ID.
The ground-floor reading room stays open.
Group rooms close at 10 pm.
Regular timings resume after examinations.

ONE-SENTENCE SUMMARY

During exams, IITGN's library and ground-floor reading room stay open until midnight from Monday; bring institute ID.

How does the model decide what matters?

Answer a question using the context

CONTEXT

IIT Gandhinagar's permanent campus is located in Palaj, on the banks of the Sabarmati River.

QUESTION

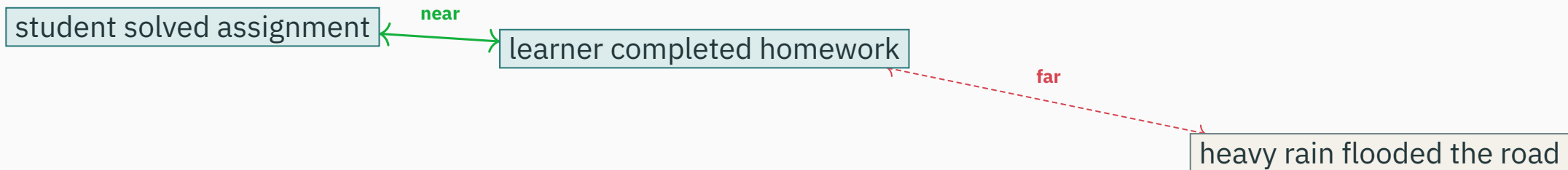
Where is IITGN's permanent campus?

ANSWER

Palaj

For this question, we need the location. Another question may need a different part of the context.

Represent similar meanings with nearby vectors



Can similar sentences have nearby vectors?

Generation has many valid answers

PROMPT

The student opened the laptop and ...

A

started coding.

C

joined class.

B

checked email.

D

saw a warning.



Several answers make sense. What should the model output?

Generation is still classification

PREFIX

The student opened the laptop and

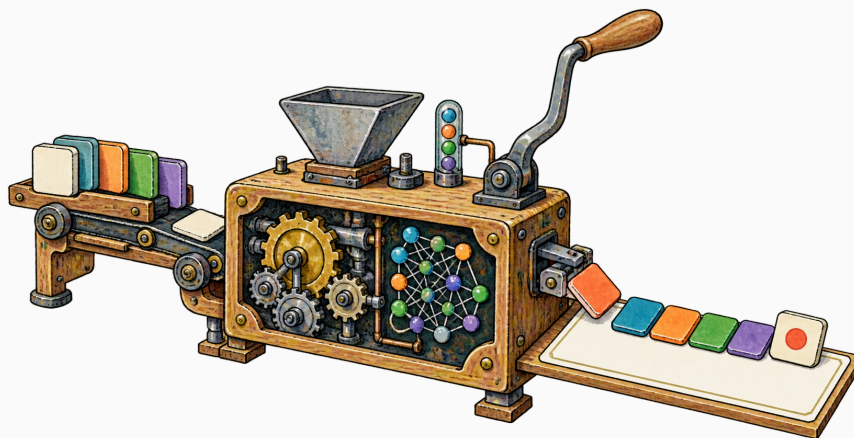


A probability for each possible next token.

Generate a name

Start with a small dataset.

**Can a neural network
generate Indian names?**



aabid

aarav

avni

naveen

vani

zeel

Worked example: Nipun Batra, Next-Token Prediction.

What exactly should we predict?

CHARACTER

transformer → t | r | a | ...

WORD

deep learning → deep | learning

SUBWORD

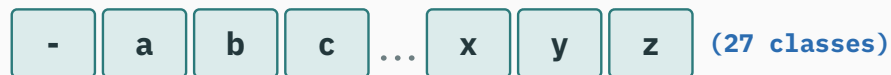
unbelievable → un | believ | able

A token is one unit the model predicts.

Start with one character per token. We can write out every training example and vector.

The task: generate an Indian name

VOCABULARY



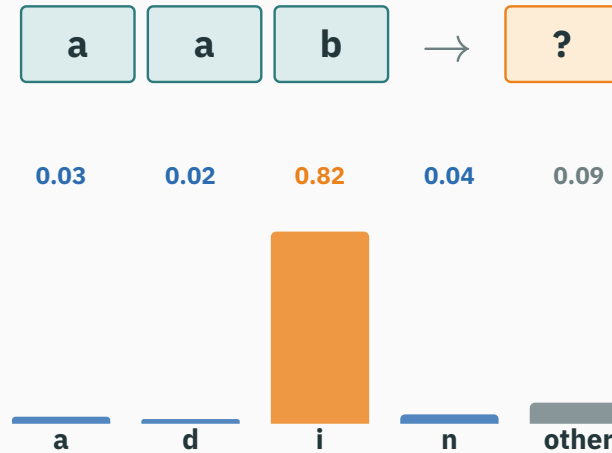
$$\mathcal{V} = \{-, a, \dots, z\}, \quad |\mathcal{V}| = 27$$

BOUNDARY SYMBOL

Put - before a name to fill the context, and after it to mark the end. In this example, the same symbol handles BOS, padding, and EOS.

Learn a probability distribution over 27 possible next characters.

Generation is repeated 27-class classification



$$f_{\theta}(a, a, b) = p_{\theta}(c_{\text{next}} \mid a, a, b)$$

Choose a context length

How much history does the first model receive?

$$k = 1$$

one previous character

$$k = 3$$

three previous characters

$$k = \text{ALL}$$

a variable-length prefix

Use $k = 3$: $p_{\theta}(x_t \mid x_{t-3:t-1})$.

This gives the MLP a fixed-size input. We will try longer contexts later.

Construct the training examples

pair	1	2	3	4	5	6
context	---	--a	-aa	aab	abi	bid
target	a	a	b	i	d	-

Six contexts, six next-character targets.

Predict the five characters in the name, then predict - after d to end it.

Slide the context window

NAME WITH BOUNDARY SYMBOLS

- - - a a b i d -

WINDOW 1



SHIFT
RIGHT



SHIFT
RIGHT



Later, causal self-attention will let us predict these targets in parallel during training.

Repeat for every name

ONE NAME

length L
 L characters + one EOS target

→

EXAMPLES

$L + 1$ context-target pairs

$$N_{\text{examples}} = \sum_{n=1}^N (L_n + 1)$$

Split the names into training, validation, and test sets **before** making the windows.

How do we turn characters into numbers?

$$\text{a a b} \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

CHARACTERS

a, b, i: categorical symbols

MLP INPUT

$x \in \mathbb{R}^d$: a vector of numbers

? $\rightarrow$

Give each character a one-hot vector.

One-hot encoding

$$\text{a a b} \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

token	1	2	3	9	...	27
a	1	0	0	0	...	0
b	0	1	0	0	...	0
i	0	0	0	1	...	0

$$\underline{o_a^T o_b = o_a^T o_i = o_b^T o_i = 0}$$

Every pair of different characters has dot product zero. These vectors identify characters; they do not tell us which characters are similar.

Learn an embedding for each character

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

EMBEDDING TABLE

$$E \in \mathbb{R}^{27 \times 2}$$

LOOKUP RULE

$$e_c = E[\text{id}(c)]$$

LEARNABLE

row	dim 1	dim 2
-	0.1	-0.4
a	1.2	0.1
b	0.2	1.1
...	...	...
z	-0.3	0.6

Backpropagation updates the rows to help predict the next token.

Look up the embedding

a a b $\rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$

token	id	selected row
a	1	$E[1] = (1.2, 0.1)$
b	2	$E[2] = (0.2, 1.1)$
i	9	$E[9] = (-0.8, 0.9)$

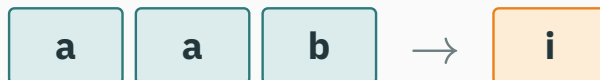
PyTorch

```
import torch
embed = torch.nn.Embedding(27, 2)
ids = torch.tensor([1, 1, 2])
rows = embed(ids)
assert rows.shape == (3, 2)
```

Each ID selects one trainable row.

Check the shapes

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$



INTEGER INPUT

$$X = (1, 1, 2)$$

shape: (3,)

EMBEDDINGS

$$E[X] = \begin{pmatrix} 1.2 & 0.1 \\ 1.2 & 0.1 \\ 0.2 & 1.1 \end{pmatrix}$$

shape: 3×2

One row per token, one column per learned feature.

Concatenate the embeddings

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

$$\begin{pmatrix} 1.2 & 0.1 \\ 1.2 & 0.1 \\ 0.2 & 1.1 \end{pmatrix} \rightarrow h_0 = [1.2, 0.1; 1.2, 0.1; 0.2, 1.1]$$

POSITION 1 · A

[1.2, 0.1]

POSITION 2 · A

[1.2, 0.1]

POSITION 3 · B

[0.2, 1.1]

$$h_0 \in \mathbb{R}^{kd} = \mathbb{R}^6$$

Why concatenate?

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

ORDER 1

$$a \ b \ i \rightarrow [e_a; e_b; e_i]$$

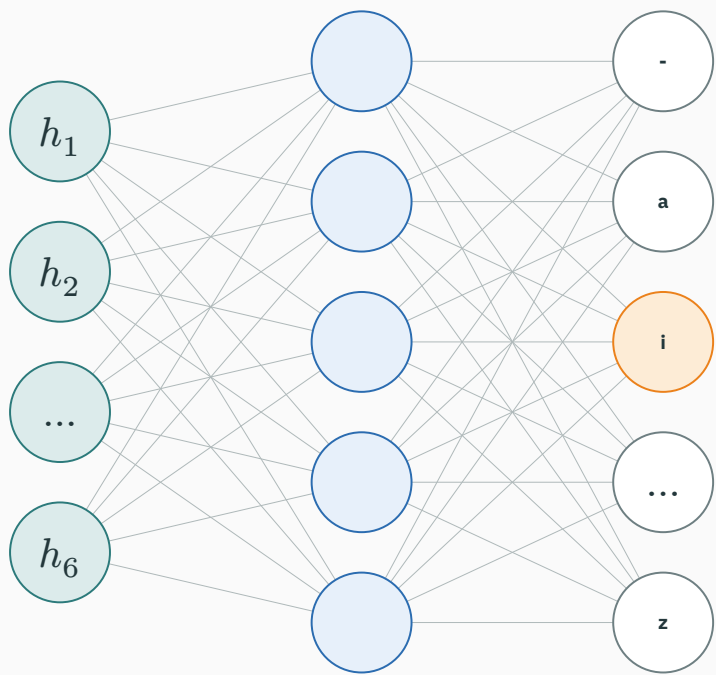
ORDER 2

$$i \ b \ a \rightarrow [e_i; e_b; e_a]$$

$$[e_a; e_b; e_i] \neq [e_i; e_b; e_a]$$

Adding gives the same result in both orders: $e_a + e_b + e_i = e_i + e_b + e_a$. Concatenation keeps each position in a separate part of the vector.

Pass the vector through an MLP



ONE HIDDEN LAYER

$$h_1 = \sigma(W_1 h_0 + b_1)$$

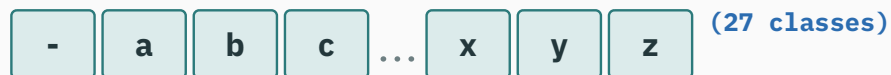
$$z = W_2 h_1 + b_2$$

$$W_1 \in \mathbb{R}^{H \times 6}, \quad W_2 \in \mathbb{R}^{27 \times H}$$

6 input numbers; 27 output scores

Why exactly 27 outputs?

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow \text{27 logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$



$$z = W_2 h_1 + b_2 \in \mathbb{R}^{27}$$

LOGIT $z_{_}$

score for boundary

LOGIT z_a

score for a

LOGIT z_i

score for i

One score for each possible next character.

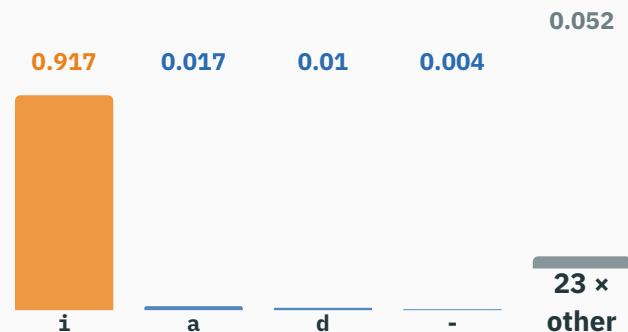
Turn the 27 scores into probabilities

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27 \text{ logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

EXAMPLE LOGITS FOR AAB

$$z_i = 5, \quad z_a = 1, \quad z_d = 0.5, \\ z_- = -0.5, \text{ and every other logit} = -1.$$

$$p_j = \frac{\exp(z_j)}{\sum_{r \in \mathcal{V}} \exp(z_r)}$$



All 27 probabilities are positive and sum to one. The other bar groups the remaining 23 characters.

Compute the loss

$$a, a, b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \xrightarrow{\text{MLP}} 27_{\text{logits}} \xrightarrow{\text{softmax}} \text{CE}(i)$$

For target i : $\ell = -\log p_{\theta}(i \mid a, a, b)$

CONFIDENT AND CORRECT

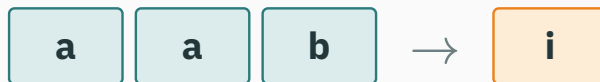
$$p(i) = 0.80$$
$$\ell = -\log(0.80) \approx 0.223$$

CONFIDENT AND WRONG

$$p(i) = 0.01$$
$$\ell = -\log(0.01) \approx 4.605$$

The more probability we give i , the smaller the loss.

The loss in PyTorch



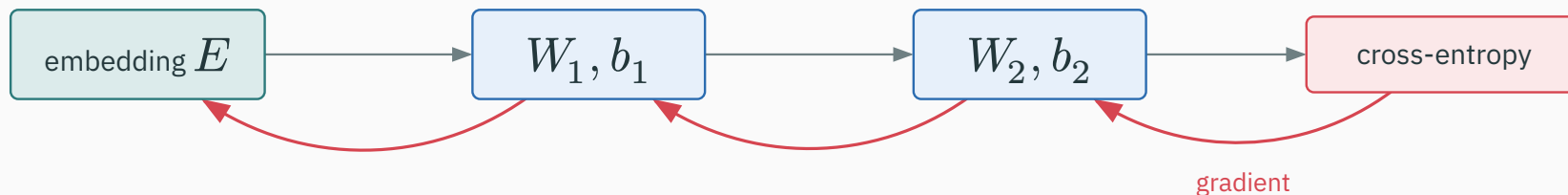
PyTorch

```
import torch
import torch.nn.functional as F
# logits has shape [1, 27]
target = torch.tensor([9]) # token i
loss = F.cross_entropy(logits, target)
loss.backward()
```

Pass the target ID to cross-entropy. Backpropagation computes gradients for the whole network.

Learn the embeddings and MLP weights

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27 \text{ logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

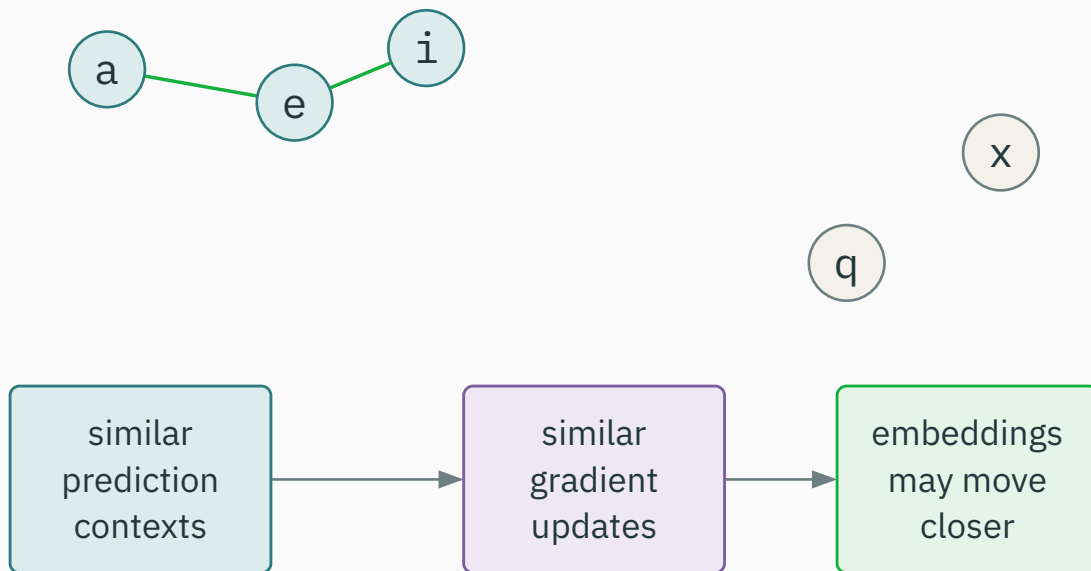


The same loss trains the embedding table and the MLP.

For $aab \rightarrow i$, we look up the rows for a and b , so those rows receive gradients. The target i does not cause an embedding lookup.

When could two embeddings become similar?

$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27 \text{ logits} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$



Sample, append, repeat

$$a \ a \ b \rightarrow E[\cdot] \rightarrow [e_a; e_a; e_b] \rightarrow \text{MLP} \rightarrow 27_{\text{logits}} \rightarrow \text{softmax} \rightarrow \text{CE}(i)$$

START



SHIFT +
SAMPLE



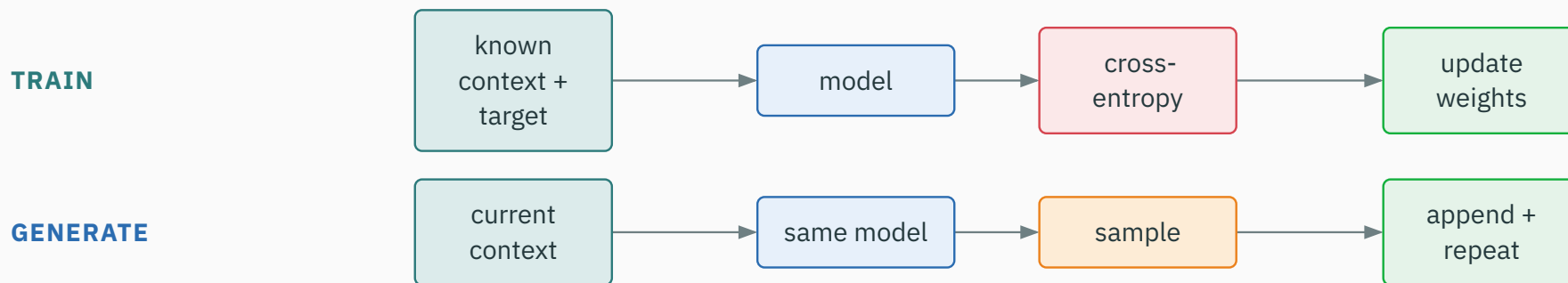
SHIFT +
SAMPLE



aab $\rightarrow$ i $\rightarrow$ abi $\rightarrow$ d $\rightarrow$ bid $\rightarrow$ -

We sampled -, so stop. The generated name is aabid.

Training and generation



Training uses the observed next token. Generation samples one.

Our character language model

$$p_{\theta}(x_t \mid x_{t-3:t-1})$$

SIZE

27 symbols and three context characters

CHARACTER-LEVEL

one character is one token

LANGUAGE MODEL

predicts the next token from previous tokens

Much larger generative models use this same next-token objective.

Neural language model: Bengio et al. (2003) · worked example: Next-Token Prediction.

Why begin with characters?

SENTENCE

deep learning is amazing

d | e | e | p | ^{SP} | l | e | a | r | n | i | n | g | ^{SP} | i | s | ...

SMALL VOCABULARY

we can write out every lookup and target

LONG SEQUENCES

even an ordinary sentence has many tokens

Character tokens

transformer → t | r | a | n | s | f | o | r | m | e | r

VOCABULARY

tiny: letters, digits, punctuation,
bytes

SEQUENCE

very long

UNKNOWN TEXT

almost none at byte/Unicode-unit
level

Small units cover many strings. They also produce long sequences.

Word tokens

deep learning is useful → deep | learning | is | useful

ONE ROOT, MANY VOCABULARY ENTRIES

walk · walks · walked · walking · walker · walkability

SHORT SEQUENCES

fewer tokens; words are easy to interpret

LARGE VOCABULARY

many entries; unseen words need handling

What happens to an unseen word?



These different words all become <UNK>.

Can we split text at spaces?

MIXED TEXT

IITGN में आज transformer lecture है :)

URL

<https://iitgn.ac.in/events?id=42>

CODE

```
scores[i] += q @ k.T
```

SCRIPTS

Devanagari + Latin

PUNCTUATION

`://?=[ ]@`

EMOJI

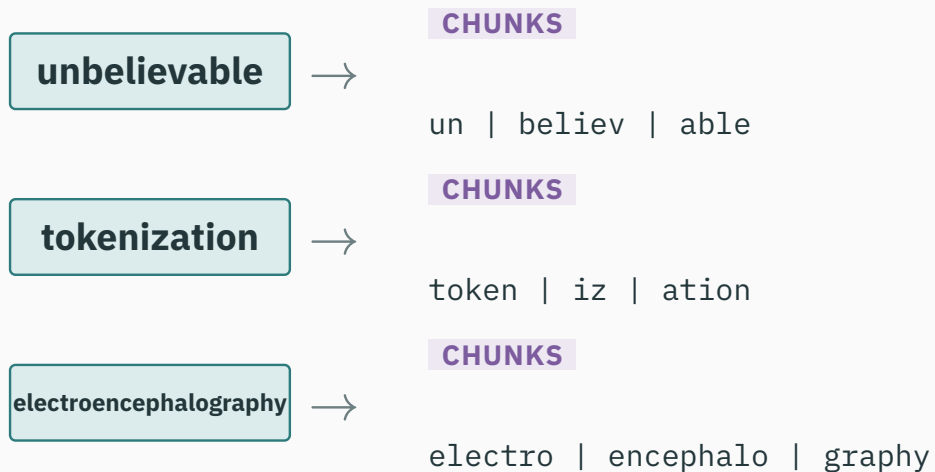
`:)` or Unicode

WHITESPACE

not a universal boundary

What counts as a word in each example?

Subword tokens



Reuse common pieces to represent both frequent and rare strings. We will leave the BPE training algorithm for later.

Choose the token size

CHARACTERS

tiny V · long T



SUBWORDS

moderate V ·
moderate T



WORDS

huge V · short T

LARGER UNITS

more meaning per displayed token

SMALLER UNITS

easier to represent unfamiliar strings

Tokenization comparison: Stanford CME 295 L1 · MIT 6.S191 L2.

Compare the trade-off

unit	V	T	unseen text	in practice
character	small	long	excellent	meaning spans many steps
subword	moderate	moderate	strong	balances V and T
word	huge	short	brittle	easy to read

Tokenization changes vocabulary size V and sequence length T .

We will use T again in attention: its score matrix has shape $T \times T$.

Tokens become integer IDs

token	id
The	17
cat	42
sat	91
on	8
the	5
mat	73

TOKEN SEQUENCE

The | cat | sat | on | the | mat

ID SEQUENCE

[17, 42, 91, 8, 5, 73]

An ID labels a row in the learned embedding table.

Use token IDs in the model

$$x_1, x_2, \dots, x_T$$

$$x_i \in \{1, 2, \dots, V\}$$

DISPLAY

We will often show whole words.

MODEL INPUT

Each displayed unit has a token ID.

OBJECTIVE

Predict the next character, subword, or word.

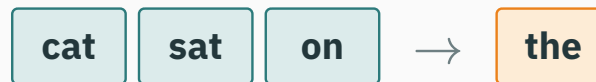
Change the tokens, keep the model

CHARACTER MODEL



embedding $\rightarrow$ concatenate $\rightarrow$ MLP $\rightarrow$ softmax

TOKEN MODEL



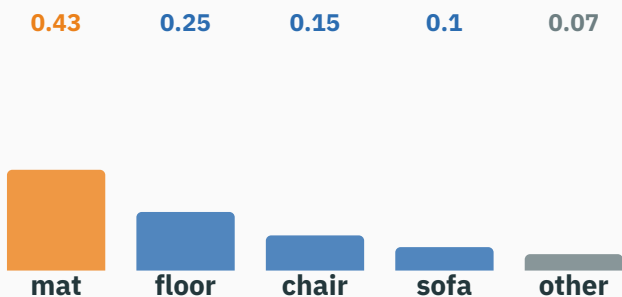
embedding $\rightarrow$ concatenate $\rightarrow$ MLP $\rightarrow$ softmax

We changed the vocabulary, the tokenizer, and the training data.

What could come next?

PREFIX

The cat sat on the ___



The training text gives us one target at each position. The model can give probability to several reasonable continuations.

Construct examples from a sentence

visible prefix	next target
<BOS>	deep
deep	learning
deep learning	is
deep learning is	fun
deep learning is fun	<EOS>

Keep the whole prefix this time. It grows by one token per row.

Write $x_1 = \text{<BOS>}$ and $x_T = \text{<EOS>}$. At position t , predict x_{t+1} .

Multiply the next-token probabilities

deep learning is fun

$$\begin{aligned} & p(\text{deep learning is fun}) \\ &= p(\text{deep}) \\ &\quad \times p(\text{learning} \mid \text{deep}) \\ &\quad \times p(\text{is} \mid \text{deep learning}) \\ &\quad \times p(\text{fun} \mid \text{deep learning is}). \end{aligned}$$

The chain rule gives this product exactly. We still need a model to turn each prefix into a prediction.

How should we represent the context?

$$\mathcal{L}(\theta) = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

TRAINING OBJECTIVE

Increase the probability of the observed next token at every position.

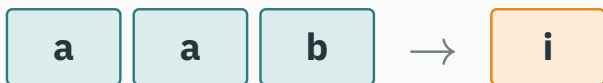
MODEL DESIGN

How do we represent a prefix that keeps getting longer?

The same loss can train different ways of representing the context.

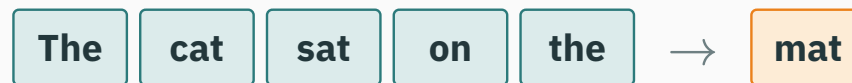
What if the context keeps growing?

THREE CHARACTERS



$(3 \times d \text{ values} \rightarrow \text{concatenate})$

GROWING PREFIX



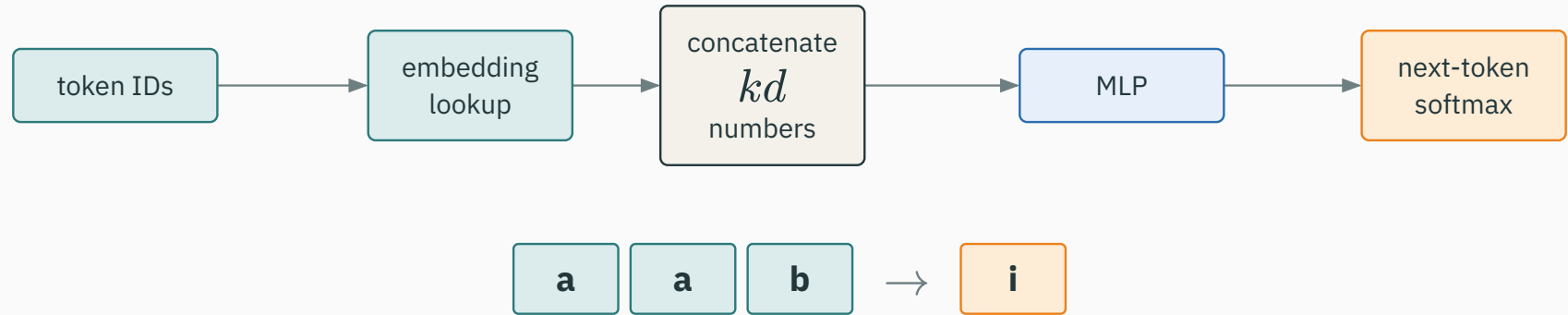
(length can keep growing)

How can the model use a longer context?

Next, compare concatenation and averaging. Then let the model choose which parts of the context to use for each query.

Lecture 2: How attention works

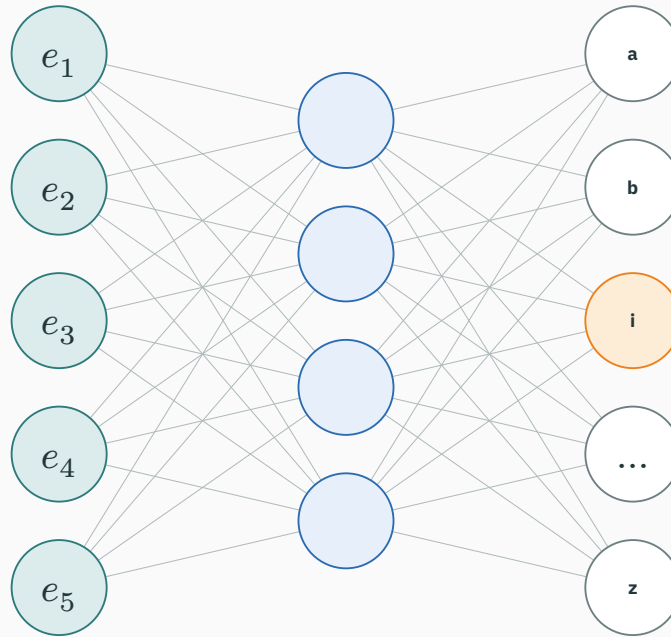
The fixed-window pipeline



| We still predict the next token. How should we combine the context vectors?

What if the window is five tokens?

Five tokens $\rightarrow$ five embedding vectors $\rightarrow$ one wider first layer



The first layer now takes $5d$ inputs instead of $3d$.

What if the window is 100 tokens?

REPRESENTATION

$$h_0 = [e_1; e_2; \dots; e_{100}] \in \mathbb{R}^{100d}$$

The input is now **100 times** the embedding width.

FIRST MLP LAYER

$$W_1 \in \mathbb{R}^{H \times 100d}$$

parameter count: $100dH + H$

If $d = 256$ and $H = 1024$, this one layer has about 26.2 million weights.

What if the window is 10,000 tokens?



10,000 embedding slots, with most omitted here

WIDTH

$10,000d$ input coordinates

PARAMETERS

$10,000dH$ first-layer weights

FIXED SIZE

the network expects exactly
10,000 slots

The window is part of the architecture

WINDOW $k = 3$

WINDOW $k = 5$

WINDOW $k = 100$

$$W_1 \in \mathbb{R}^{H \times 3d}$$

$$W_1 \in \mathbb{R}^{H \times 5d}$$

$$W_1 \in \mathbb{R}^{H \times 100d}$$

Change the window, and we must change the network.

Which context words help?

I poured the coffee from the pot into the cup because the ---

CONTINUATION A

cup was empty

Which noun matters most?

CONTINUATION B

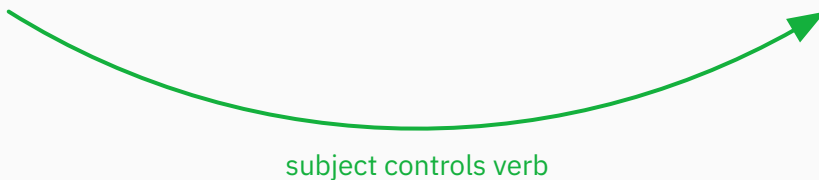
coffee was cold

Which noun matters most?

| Different predictions need different words from the context.

Useful evidence can be far away

The keys to the old wooden cabinet are missing



keys is plural → are. The nearer noun, cabinet, is singular.

Who does she refer to?

Alice texted Priya after **she** finished the report.

CANDIDATE 1

she → Alice

CANDIDATE 2

she → Priya

This sentence alone does not tell us who she is. Attention cannot resolve it without more evidence.

A lookup embedding cannot see context

RIVER

e_{bank}

FINANCE

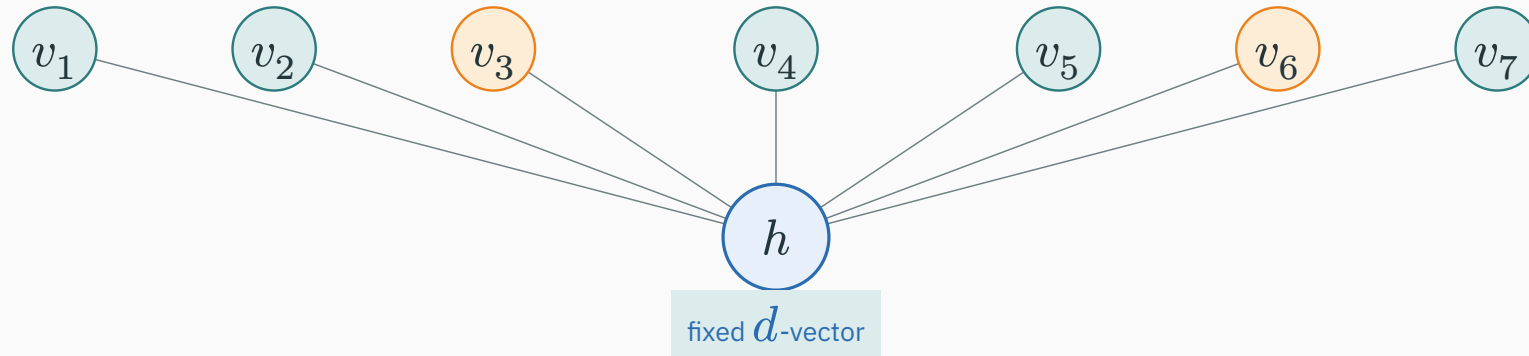
They sat on the **bank** of the Narmada.

same table row

She deposited money in the **bank**.

We want the vector for bank to depend on the sentence.

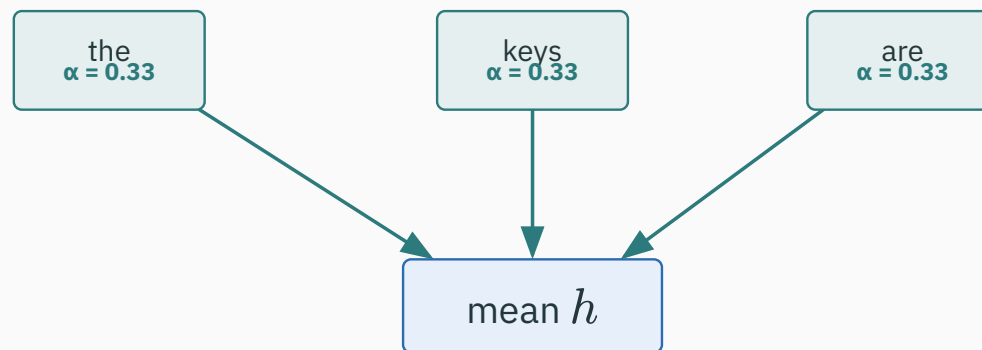
What do we want from a context summary?



Combine any number of token vectors into one d -dimensional vector.

First try: average the token vectors

$$h = \frac{1}{T} \sum_{j=1}^T v_j$$



| The output has d coordinates, however many vectors we average.

Average one tiny example



$$v_a = \begin{pmatrix} 2 & 0 \end{pmatrix}, \quad v_b = \begin{pmatrix} 0 & 2 \end{pmatrix}$$

$$h = \frac{1}{3}((2 \ 0) + (2 \ 0) + (0 \ 2))$$

$$\underline{h = \begin{pmatrix} \frac{4}{3} & \frac{2}{3} \end{pmatrix}}$$

The width stayed 2, even though three vectors went in.

The mean gives every token the same weight

keys to the cabinet are missing

0.167

0.167

0.167

0.167

0.167

0.167



Every word gets $\frac{1}{6}$. We cannot give keys more weight than the other words.

The mean also forgets order

dog bites man

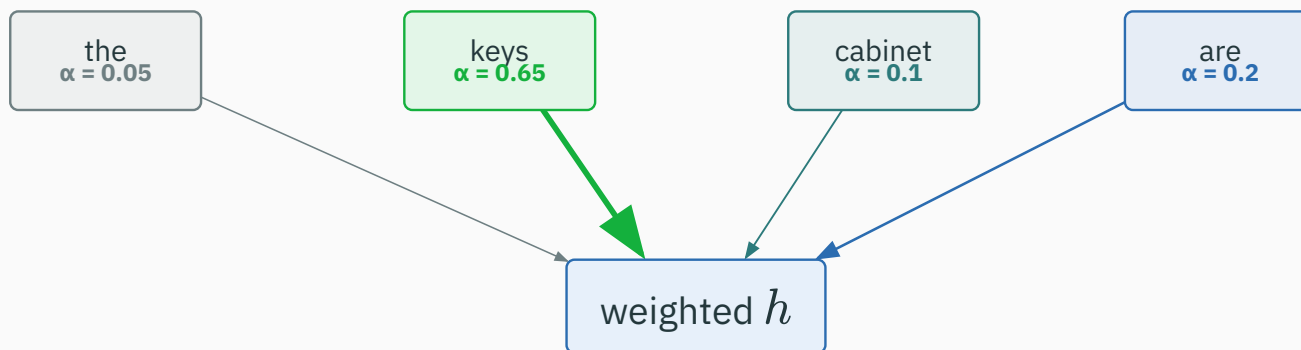
man bites dog

$$\text{mean}(e_{\text{dog}}, e_{\text{bites}}, e_{\text{man}}) = \text{mean}(e_{\text{man}}, e_{\text{bites}}, e_{\text{dog}})$$

Both sequences have the same mean. Self-attention will need position information too. We will add it before computing Q, K, and V.

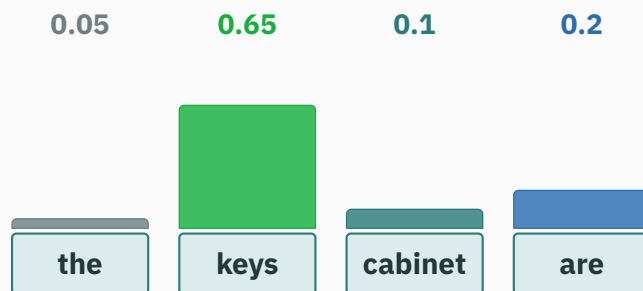
Next try: use a weighted average

$$h = \sum_{j=1}^T \alpha_j v_j$$



Each input contributes a different amount. The output still has d coordinates.

Read the weights as mixing amounts



Here, keys supplies most of the mixture.

These weights show how one head mixes its values. They do not explain the whole model.

Who should choose the weights?

Where should $\alpha_1, \dots, \alpha_T$ come from?

A · POSITION

always prefer nearby tokens

B · IDENTITY

give each word a fixed importance

C · QUERY

score each source for this query

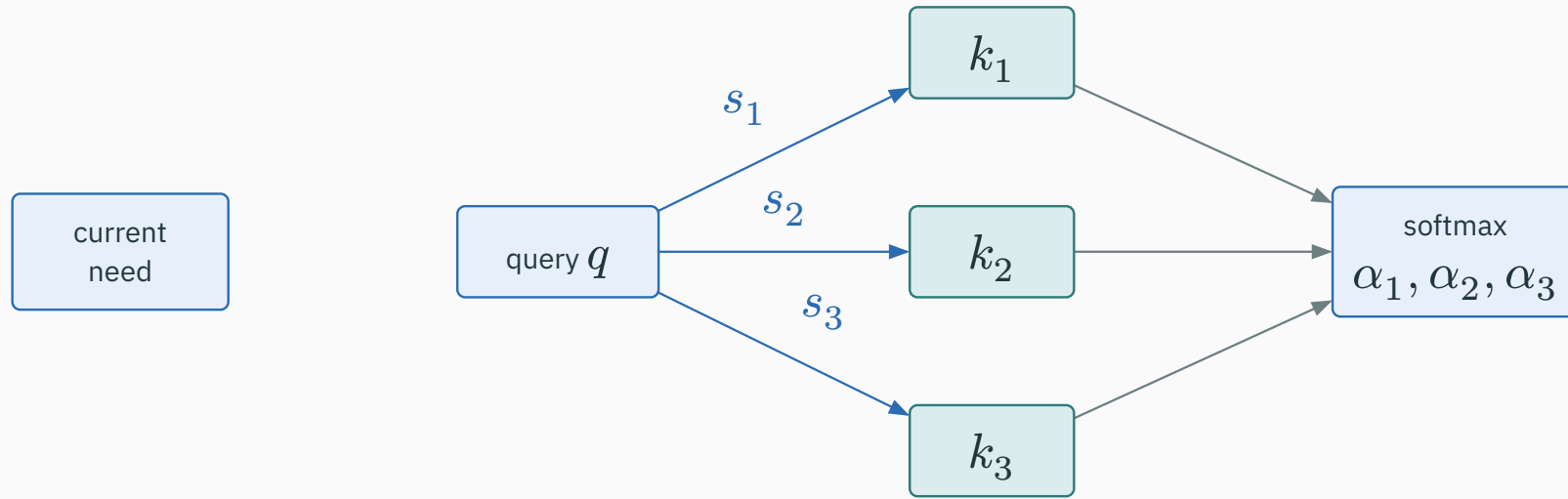
Use C: compute each weight from the query and the source.

One fixed set of weights is not enough

	question or task	useful source
sentiment	The plot was slow but the ending was wonderful	wonderful
pronoun	Alice thanked Priya because she helped	Alice Or Priya
next token	The keys to the cabinet are	keys

Which word helps depends on the question we are answering.

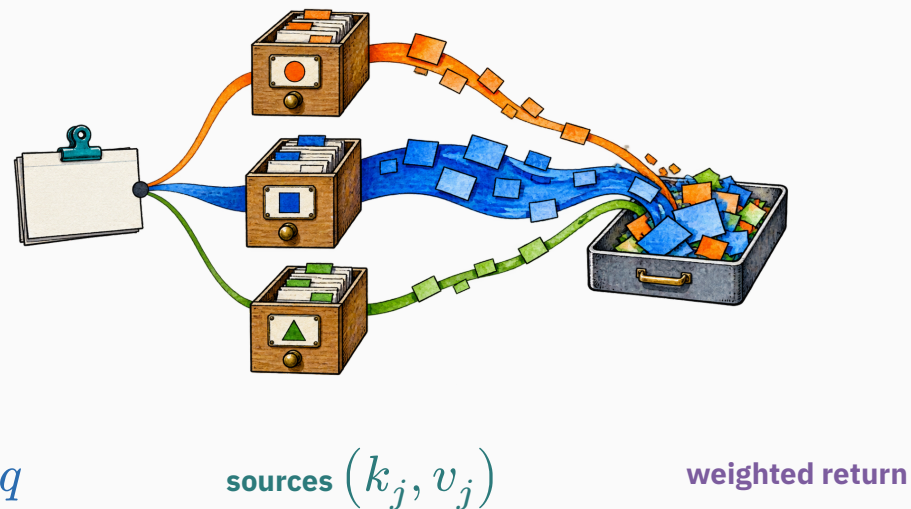
Compute the weights from a query



Compare the query with each key, then apply softmax to the scores.

Attention is a soft lookup

request $\rightarrow$ query q
address $\rightarrow$ key k_j
match $\rightarrow$ score $q^\top k_j$
payload $\rightarrow$ value v_j
return $\rightarrow \sum_j \alpha_j v_j$



j names the source. α_j says how much of its value enters the return.

Query: what am I looking for?

QUERY

q

what this token is looking for

EXAMPLES

- Which earlier noun controls this verb?
- Which entity does this pronoun refer to?
- What topic helps predict the next token?

The network makes *q* from the current token state.

Key: when should this source match?

KEY

k

match features carried by one source

source 1

k_1

source 2

k_2

source 3

k_3

$$s_j = q^\top k_j$$

The small j is the source number. A larger score means source j matches the query better.

Value: what should this source send?

VALUE

v_j

payload carried by source j

WHY SEPARATE IT FROM THE KEY?

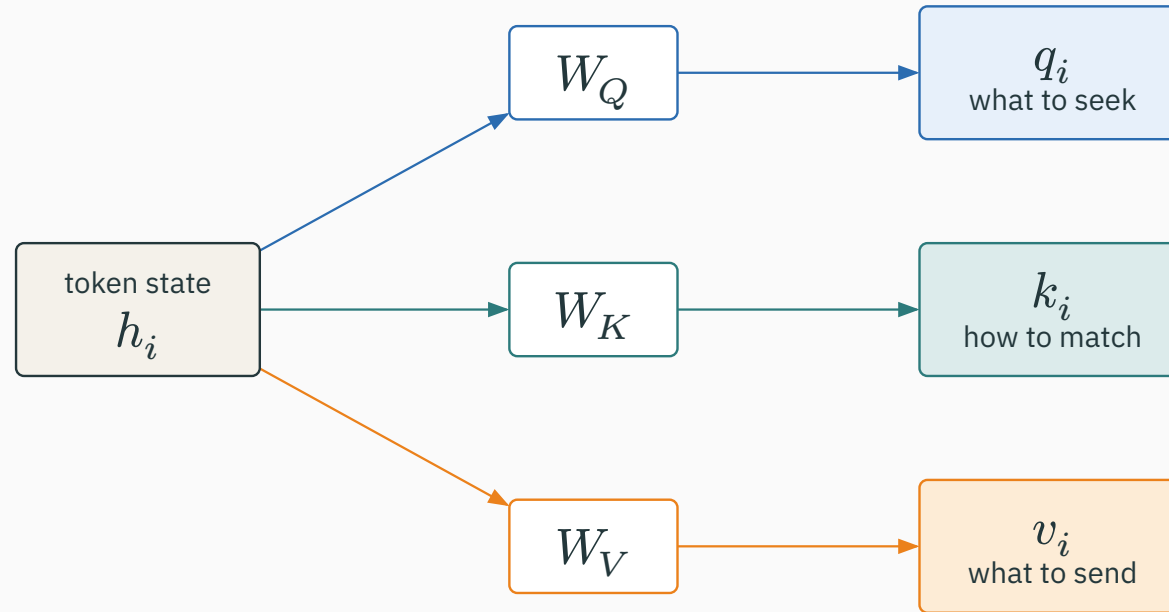
Matching and sending are different jobs.

Address: “I am a plural noun.”

Payload: semantic and syntactic information.

Compare queries with keys. Use the resulting weights to mix values.

Make a query, key, and value

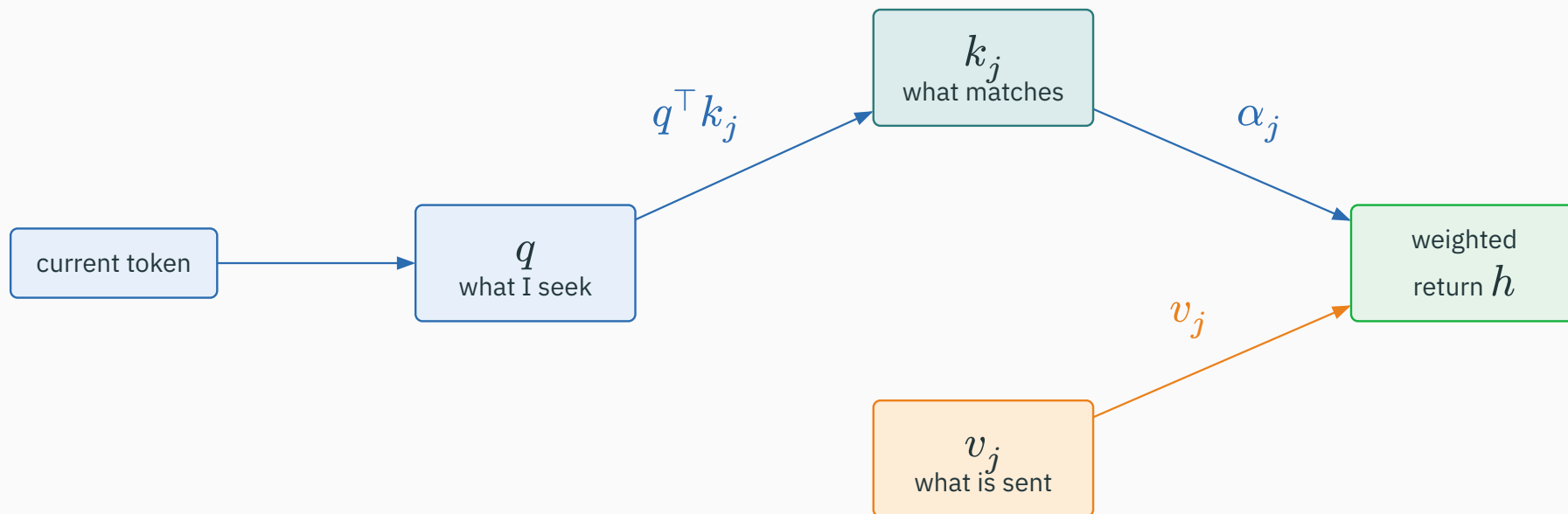


$$q_i = h_i W_Q, \quad k_i = h_i W_K, \quad v_i = h_i W_V$$

i is the token position. Multiply its vector by three learned matrices.

Compute attention for one query

For current token i , write $q = q_i$. Compare it with sources $j = 1, \dots, T$.



$$(s_j = q^\top k_j)$$

$\rightarrow$

$$(\alpha = \text{softmax}(s))$$

$\rightarrow$

$$(h = \sum_j \alpha_j v_j)$$

Write the three steps in one line

$$\text{Attention}(q, K, V) = \text{softmax}(qK^\top)V$$

ONE QUERY

q : one row
for the current token

MEMORY

K, V : T rows
one per source token

| Rows are queries; columns are source tokens and their keys.

For one query, softmax runs across the T key columns.

One query in PyTorch

PyTorch

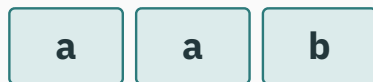
```
# q: [d], K: [T, d], V: [T, dv]
scores = q @ K.T
weights = scores.softmax(dim=-1)
out = weights @ V
```

Compare with keys, apply softmax, then mix values.

Rows are queries; columns are source tokens and their keys.

Attention on the character example

CONCATENATION



$$[e_a; e_a; e_b] \in \mathbb{R}^{3d}$$

Three fixed slots.

ATTENTION



Use the query from **b** and a key and value from each visible token.

The output is one d -dimensional vector.

We still predict **i from a a b, now using attention to combine the context.**

Pick tiny vectors we can check by hand

query
key
value

a₁

$$\begin{aligned} k_1 &= \begin{pmatrix} 1 & 0 \end{pmatrix} \\ v_1 &= \begin{pmatrix} 2 & 0 \end{pmatrix} \end{aligned}$$

a₂

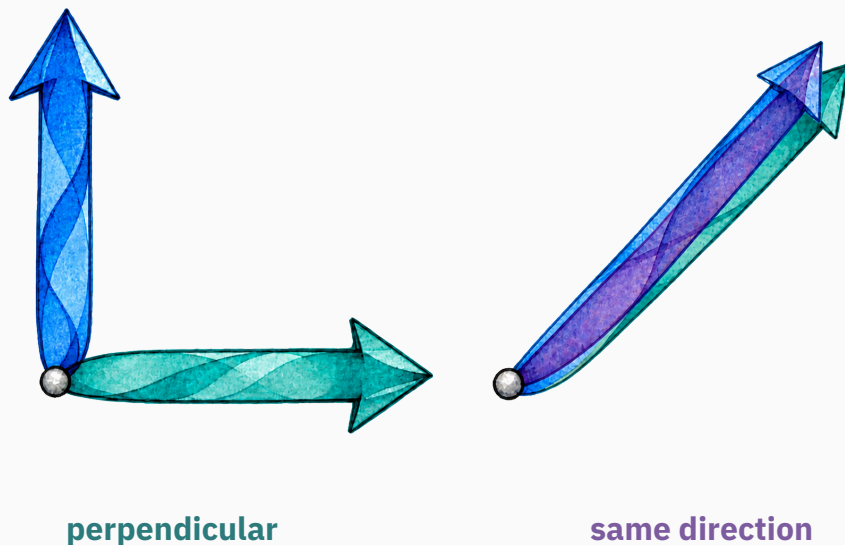
$$\begin{aligned} k_2 &= \begin{pmatrix} 1 & 0 \end{pmatrix} \\ v_2 &= \begin{pmatrix} 2 & 0 \end{pmatrix} \end{aligned}$$

b₃

$$\begin{aligned} q_b &= \begin{pmatrix} 0 & 1 \end{pmatrix} \\ k_3 &= \begin{pmatrix} 0 & 1 \end{pmatrix} \\ v_3 &= \begin{pmatrix} 0 & 2 \end{pmatrix} \end{aligned}$$

These are chosen numbers for a worked example. Leave out $\frac{1}{\sqrt{d_k}}$ for now; we will include it later.

What does the dot product notice?



QUERY STAYS FIXED

$$q = (0 \ 1)$$

PERPENDICULAR

$$k_1 = (1 \ 0)$$

$$q^\top k_1 = 0$$

SAME DIRECTION

$$k_3 = (0 \ 1)$$

$$q^\top k_3 = 1$$

| For unit vectors, the dot product measures directional agreement. Vector length matters in general.

Step 1: compare the query with each key

SOURCE a_1

$$\begin{aligned}s_1 &= q_b^\top k_1 \\ &= (0 \ 1) \cdot (1 \ 0) \\ &= 0 \times 1 + 1 \times 0 = 0\end{aligned}$$

SOURCE a_2

$$\begin{aligned}s_2 &= q_b^\top k_2 \\ &= (0 \ 1) \cdot (1 \ 0) \\ &= 0 \times 1 + 1 \times 0 = 0\end{aligned}$$

SOURCE b_3

$$\begin{aligned}s_3 &= q_b^\top k_3 \\ &= (0 \ 1) \cdot (0 \ 1) \\ &= 0 \times 0 + 1 \times 1 = 1\end{aligned}$$

$$s = (0 \ 0 \ 1)$$

Step 2: turn scores into weights

$$\alpha = \text{softmax}(0, 0, 1) = \frac{1}{e^0 + e^0 + e^1} (e^0 \ e^0 \ e^1) = \frac{1}{2 + e} (1 \ 1 \ e).$$

$$\alpha \approx (0.212 \ 0.212 \ 0.576)$$

0.212 **0.212** **0.576**



$0.212 + 0.212 + 0.576 = 1.000$: one distribution across key columns.

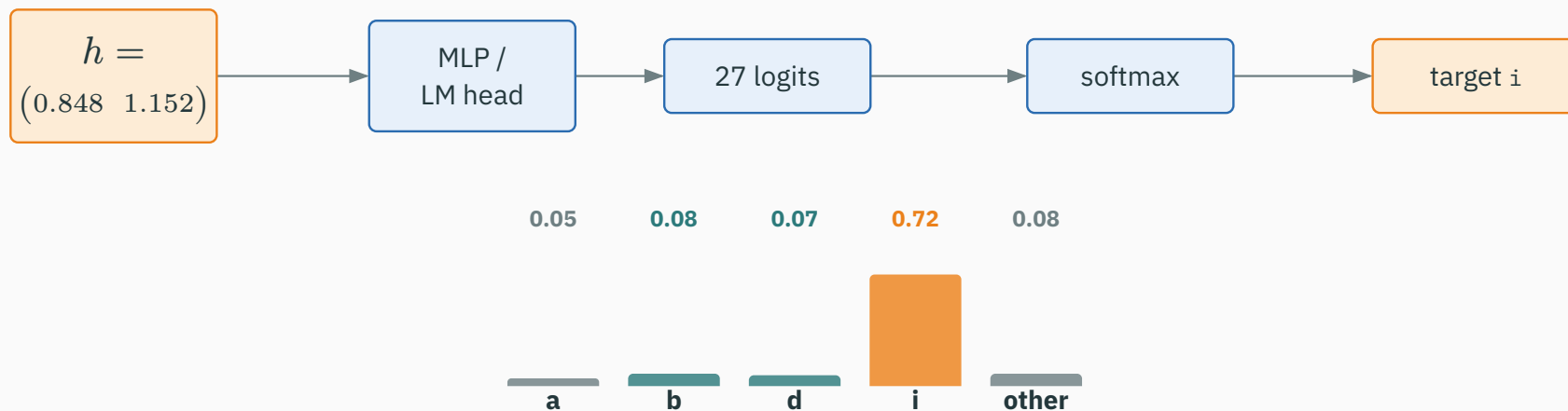
Step 3: mix the values

$$\begin{aligned}h &= 0.212v_1 + 0.212v_2 + 0.576v_3 \\&= 0.212(2 \ 0) + 0.212(2 \ 0) + 0.576(0 \ 2) \\&= (0.424 \ 0) + (0.424 \ 0) + (0 \ 1.152)\end{aligned}$$

$$\underline{h = (0.848 \ 1.152) \in \mathbb{R}^2}$$

This mixture differs from each of the three input values.

The context vector goes back to the language model



| We changed the context vector. The loss is still $-\log p_{\theta}(i \mid a \ a \ b)$.

What if we change the query?

Keep the keys and values. Use $q' = (1 \ 0)$.

OLD QUERY q_b

scores $(0 \ 0 \ 1)$

weights $(.212 \ .212 \ .576)$

output $(.848 \ 1.152)$

NEW QUERY q'

scores $(1 \ 1 \ 0)$

weights $(.422 \ .422 \ .155)$

output $(1.688 \ .310)$

Changing only the query changed both the weights and the output.

Why a soft lookup?

HARD

choose $j^* = \arg \max_j s_j$
return only v_{j^*}

One winner.

HARD LOOKUP

The $\arg \max$ selects one value.

SOFT

$\alpha = \text{softmax}(s)$
return $\sum_j \alpha_j v_j$

A smooth mixture.

SOFT LOOKUP

Differentiable weights let several sources contribute, including uncertain matches.

Which parts are learned?

QUERY PROJECTION

$$q_i = h_i W_Q$$

learns what to seek

KEY PROJECTION

$$k_i = h_i W_K$$

learns how to match

VALUE PROJECTION

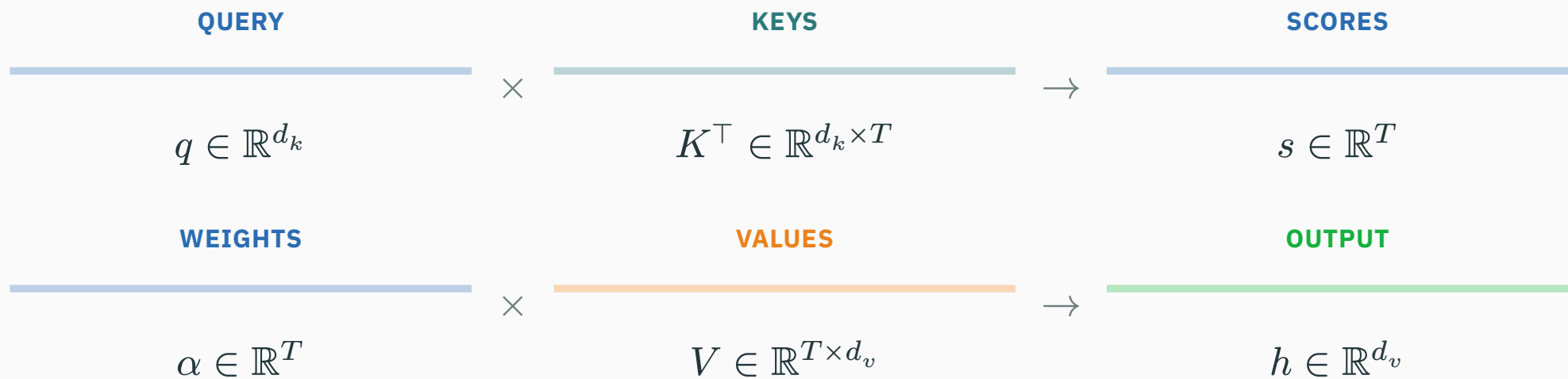
$$v_i = h_i W_V$$

learns what to send

We learn W_Q, W_K, W_V . We recompute scores for every sentence.

Backpropagation learns the three projections together with the language model.

Shapes for one query



Why should only the final token ask?

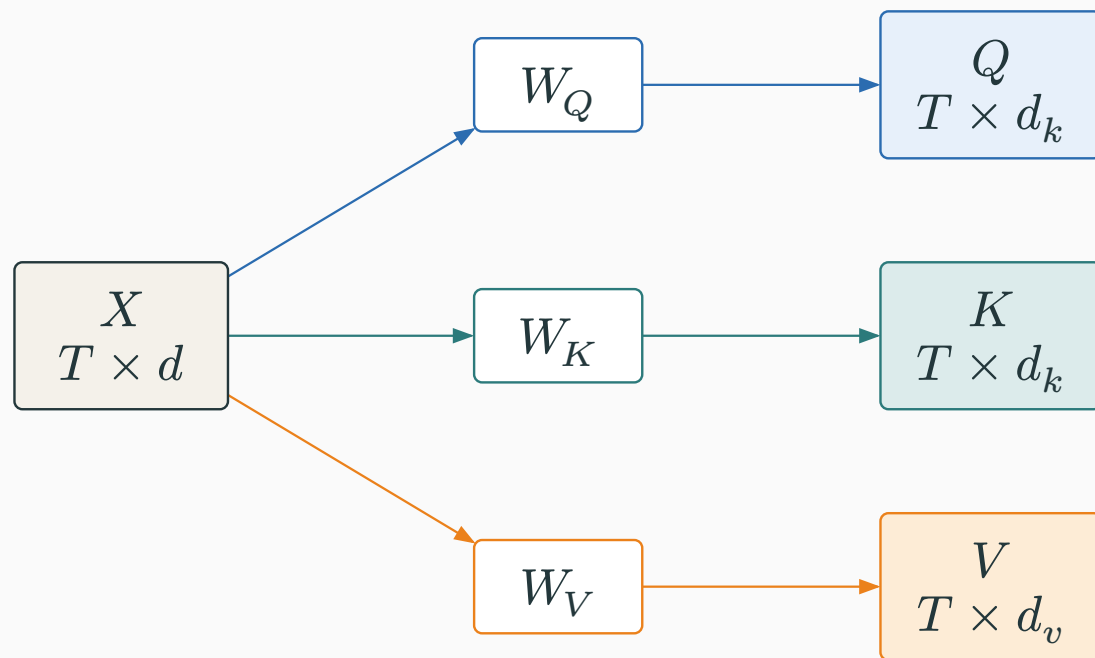
	a ₁	a ₂	b ₃
	a ₁	a ₂	b ₃
a ₁	•	•	•
a ₂	•	•	•
b ₃	•	•	•

each row is one query · each column is one possible source

Compute a query at every position and compare it with all the keys.

When the sequence supplies its own queries, keys, and values, this is **self-attention**.

Let every token make Q, K, and V



| Each projection changes the vector width. We still have one row per token.

Compare every query with every key

For the toy sequence, use

$$Q = K = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

$$S = QK^{\top} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$S_{ij} = q_i^{\top} k_j$$

	k ₁ :a	k ₂ :a	k ₃ :b
q ₁ :a	1	1	0
q ₂ :a	1	1	0
q ₃ :b	0	0	1

| Rows are queries; columns are source tokens and their keys.

Softmax each query row

$$A_{i,:} = \text{softmax}(S_{i,:})$$

	k ₁ :a	k ₂ :a	k ₃ :b
q ₁ :a	.422	.422	.155
q ₂ :a	.422	.422	.155
q ₃ :b	.212	.212	.576

row sums: ≈ 1.000 (rounded) softmax axis: key columns

Mix values for every query

$$\text{With } V = \begin{pmatrix} 2 & 0 \\ 2 & 0 \\ 0 & 2 \end{pmatrix},$$

$$O = AV = \begin{pmatrix} .422 & .422 & .155 \\ .422 & .422 & .155 \\ .212 & .212 & .576 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}.$$

$$O \approx \begin{pmatrix} 1.688 & .310 \\ 1.688 & .310 \\ .848 & 1.152 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

Row i of O uses the weights computed for query i .

The full self-attention formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

PyTorch

```
Q, K, V = X @ Wq, X @ Wk, X @ Wv
scores = Q @ K.T / K.size(-1) ** 0.5
weights = scores.softmax(dim=-1)
O = weights @ V
```

One sequence, without a batch dimension. Each row is a query; dim=-1 runs across keys.

Rows are queries; columns are source tokens and their keys.

Each output vector depends on its context

BEFORE ATTENTION

occurrence 1: e_a
occurrence 2: e_a
same vocabulary-table row

→

AFTER ATTENTION

$$o_i = \sum_j A_{ij} v_j$$

depends on the query at i and its allowed
sources

| Position information will let two copies of a produce different queries. We have not added it yet.

Attention alone does not know order

Let P permute the token rows. Without a position signal or fixed mask,

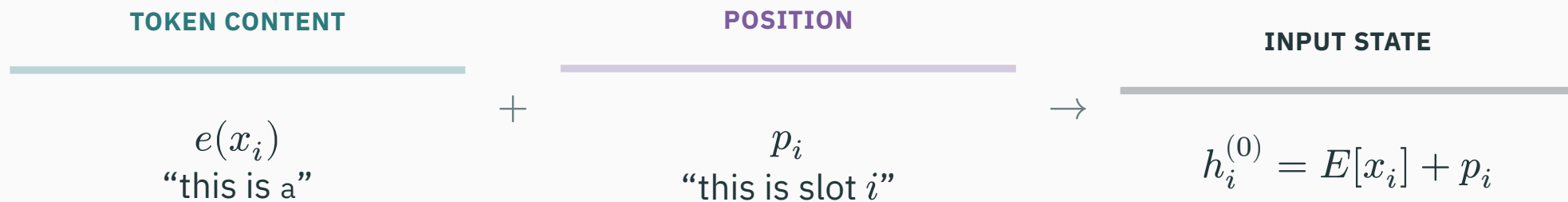
$$\text{Attn}(PX) = P \text{ Attn}(X)$$

dog **bites** **man**

man **bites** **dog**

Reordering the tokens only reorders the outputs. The vectors contain no information about which token came first.

Add position before making Q, K, and V



$$q_i = h_i^{(0)} W_Q, \quad k_i = h_i^{(0)} W_K, \quad v_i = h_i^{(0)} W_V$$

| Addition keeps the same width and makes position available to Q, K, and V.

Two simple ways to add position

LEARNED TABLE

$$p_i = P[i]$$

Learned from data.

A table lookup for each position.

Only the trained positions have entries.

SINUSOIDAL

$$p_{i,2r} = \sin\left(\frac{i}{10000^{2\frac{r}{d}}}\right)$$

$$p_{i,2r+1} = \cos\left(\frac{i}{10000^{2\frac{r}{d}}}\right)$$

Deterministic.

Defined beyond the training positions.

| Both add order. Later models often use relative or rotary position instead.

Can a training token peek at the answer?

input prefix: a a target: **b**

ALLOWED

a_2 reads positions 1 and 2

CHEATING

a_2 reads position 3 = target b

If the predictor can already see b, it can simply copy the answer.

Without a mask, future tokens leak in

Sequence: ---aabid-

	1:-	2:-	3:-	4:a	5:a	6:b	7:i	8:d	9:-
1:-	•	•	•	•	•	•	•	•	•
2:-	•	•	•	•	•	•	•	•	•
3:-	•	•	•	•	•	•	•	•	•
4:a	•	•	•	•	•	•	•	•	•
5:a	•	•	•	•	•	•	•	•	•
6:b	•	•	•	•	•	•	•	•	•
7:i	•	•	•	•	•	•	•	•	•
8:d	•	•	•	•	•	•	•	•	•
9:-	•	•	•	•	•	•	•	•	•

rows = query positions · columns = key positions · row-softmax across columns

Every cell above the diagonal is a leak: query i can read a future token $j > i$.

Draw the causal mask

	1:-	2:-	3:-	4:a	5:a	6:b	7:i	8:d	9:-
1:-	•	×	×	×	×	×	×	×	×
2:-	•	•	×	×	×	×	×	×	×
3:-	•	•	•	×	×	×	×	×	×
4:a	•	•	•	•	×	×	×	×	×
5:a	•	•	•	•	•	×	×	×	×
6:b	•	•	•	•	•	•	×	×	×
7:i	•	•	•	•	•	•	•	×	×
8:d	•	•	•	•	•	•	•	•	×
9:-	•	•	•	•	•	•	•	•	•

rows = queries · columns = keys · row-softmax runs only over allowed columns

$j \leq i$: allowed source, mask value 0

$j > i$: future source, mask value $-\infty$

Mask future scores before softmax

For query row 3 in a four-token sequence:

SCORES $S_{3,:}$		MASK $M_{3,:}$		MASKED SCORES
	+		→	
(0.2 1.0 0.4 2.7)		(0 0 0 $-\infty$)		(0.2 1.0 0.4 $-\infty$)

$$\text{softmax}(0.2, 1.0, 0.4, -\infty) \approx (.225 \ .500 \ .275 \ 0)$$

$\exp(-\infty) = 0$: forbidden keys receive exactly zero probability.

The causal mask in PyTorch

PyTorch

```
scores = Q @ K.T / K.size(-1) ** 0.5
mask = torch.triu(torch.ones(T, T), diagonal=1).bool()
scores = scores.masked_fill(mask, float('-inf'))
weights = scores.softmax(dim=-1)
out = weights @ V
```

One sequence; batch dimension omitted. Mask scores first, then softmax, then mix values.

Rows are queries; columns are source tokens and their keys.

Read a masked attention matrix

	k_1	k_2	k_3	k_4
q_1	1.00	0	0	0
q_2	.45	.55	0	0
q_3	.225	.500	.275	0
q_4	.10	.20	.30	.40

(row 1 sees 1)

(row 2 sees 1 and 2)

(row t sees 1 to t)

Every row sums to one across its allowed key columns.

One forward pass predicts every next token

INPUT	-	-	-	a	a	b	i	d
STATE	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
TARGET	-	-	a	a	b	i	d	-

state h_t predicts shifted target x_{t+1}

| Each row sees only its prefix, but all rows are still computed together.

Predict all six targets in parallel

FIXED-WINDOW MLP

--- → a
--a → a
-aa → b
aab → i
abi → d
bid → -



CAUSAL SELF-ATTENTION

One sequence: ---aabcd

take losses at predictor positions 3 to 8

targets: a a b i d -

The six targets are unchanged. Each row now uses its whole allowed prefix.

Why scale the dot product?

If coordinates are roughly independent with unit variance,

$$q^\top k = \sum_{r=1}^{d_k} q_r k_r, \quad \text{Var}(q^\top k) \approx d_k.$$

UNSCALED

large $d_k \rightarrow$ large logits $\rightarrow$ near one-hot softmax
 $\rightarrow$ tiny gradients

SCALED

$q^\top \frac{k}{\sqrt{d_k}}$ keeps score variance roughly constant

Divide by $\sqrt{d_k}$ to keep the scores at a manageable scale.

Check the attention calculation

1 • ROW SUMS

.2	.3	.5
----	----	----

$$\sum_j A_{ij} = 1$$

test A.sum(dim=-1)

2 • FUTURE STAYS ZERO

	1	2	3
1	•	×	×
2	•	•	×
3	•	•	•

inspect the upper triangle

3 • MATRIX SHAPES

$$\underbrace{(T \times d_k)(d_k \times T)}_{QK^\top}$$

$$\rightarrow T \times T$$

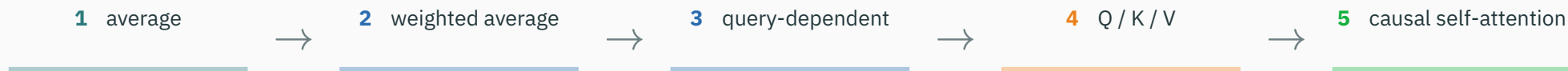
$$\underbrace{(T \times T)(T \times d_v)}_{AV}$$

$$\rightarrow T \times d_v$$

write every multiplication

Rows are queries. Columns are keys. Softmax runs across each row.

What we built today



We still predict the next token. Each position now combines its context using attention weights.

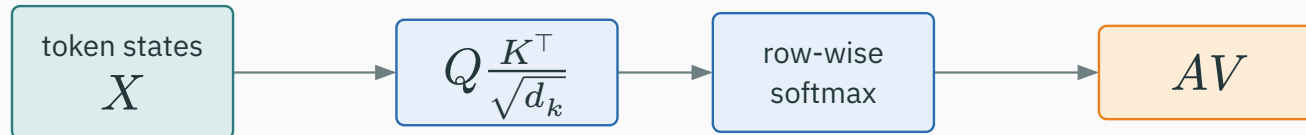
Next: what if each token asks several different questions?

Lecture 3: From attention to the Transformer

Recall one attention head

Each token gets a weighted combination of the visible values

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$



One head makes one set of Q , K , and V vectors. Each row uses its own weights to combine the values it is allowed to see.

Why use more than one head?

Priya opened notebook ; she smiled

Several parts of this sentence could help us predict what comes after `smiled`.

NEARBY

What just happened?

WHO?

Which earlier person
does she refer to?

STRUCTURE

Where did one clause end
and another begin?

Can we learn a separate set of attention weights for each?

Give each token several attention heads

HEAD 1

what is nearby?

might use nearby tokens

HEAD 2

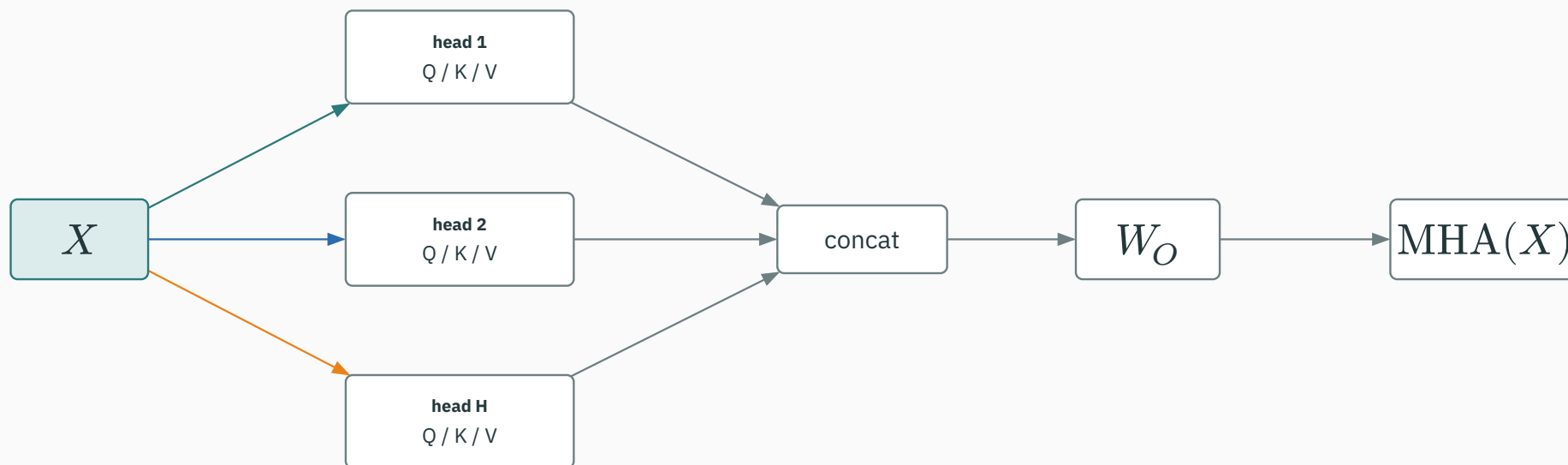
which person?

might connect distant tokens

HEAD 3

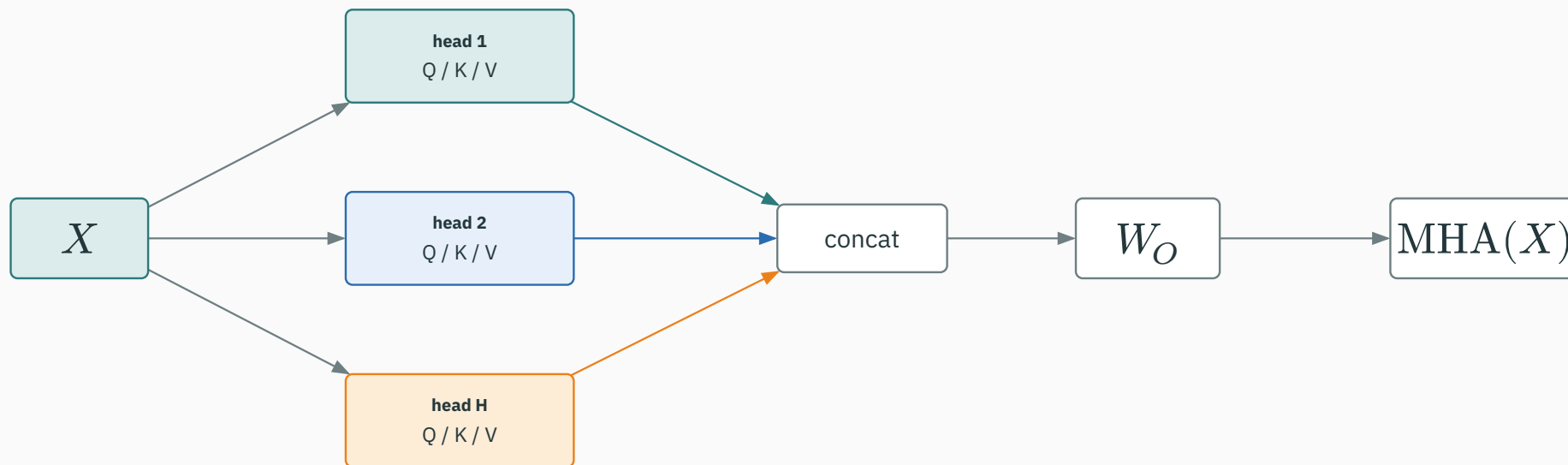
where is the boundary?

might use punctuation



Each head learns its own projections

$$Q_h = XW_Q^{(h)}, \quad K_h = XW_K^{(h)}, \quad V_h = XW_V^{(h)}$$



Every head receives the same X and learns its own weights

One head may look nearby

In this example, the row for `smiled` puts most of its weight on nearby `she`.

keys →	Priya	opened	notebook	;	she	smiled
smiled →	.05	.08	.07	.04	.66	.10

| Rows are queries; columns are source tokens and their keys.

| Softmax runs across the six key columns in this row. The weights sum to 1.

Another head may look back to a person

A second head might connect `she` with `Priya`. Here is one possible row of weights.

keys →	Priya	opened	notebook	;	she	smiled
she →	.61	.12	.10	.07	.10	0

| Rows are queries; columns are source tokens and their keys.

| `smiled` is in the future for this row. The causal mask makes its attention weight exactly zero.

A third head may notice a boundary

For a third head, the row for `smiled` might put most of its weight on the semicolon.

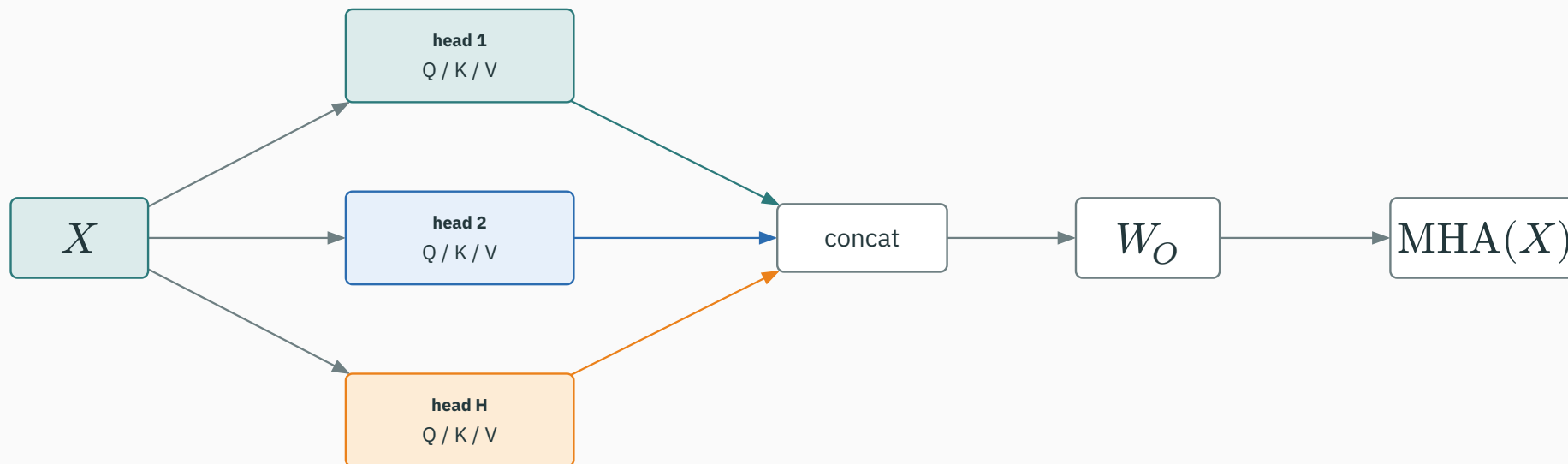
keys →	Priya	opened	notebook	;	she	smiled
smiled →	.07	.08	.09	.58	.13	.05

| Rows are queries; columns are source tokens and their keys.

| We chose these weights to illustrate possible patterns. Training learns the heads across many examples; a head need not have one clear role.

The heads do the same calculation separately

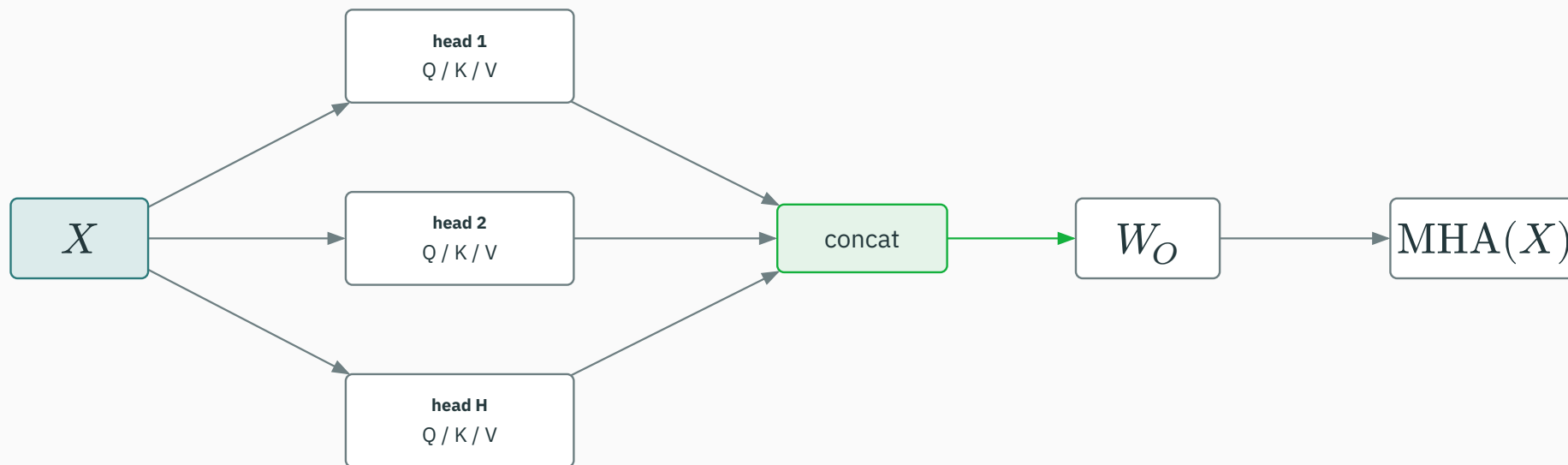
$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h) = \text{softmax}\left(Q_h \frac{K_h^\top}{\sqrt{d_k}} + M\right) V_h$$



$$(Q_h : T \times d_k) \times (K_h^\top : d_k \times T) \rightarrow (A_h : T \times T) \times (V_h : T \times d_v)$$

$$\text{head}_h \in \mathbb{R}^{T \times d_v}$$

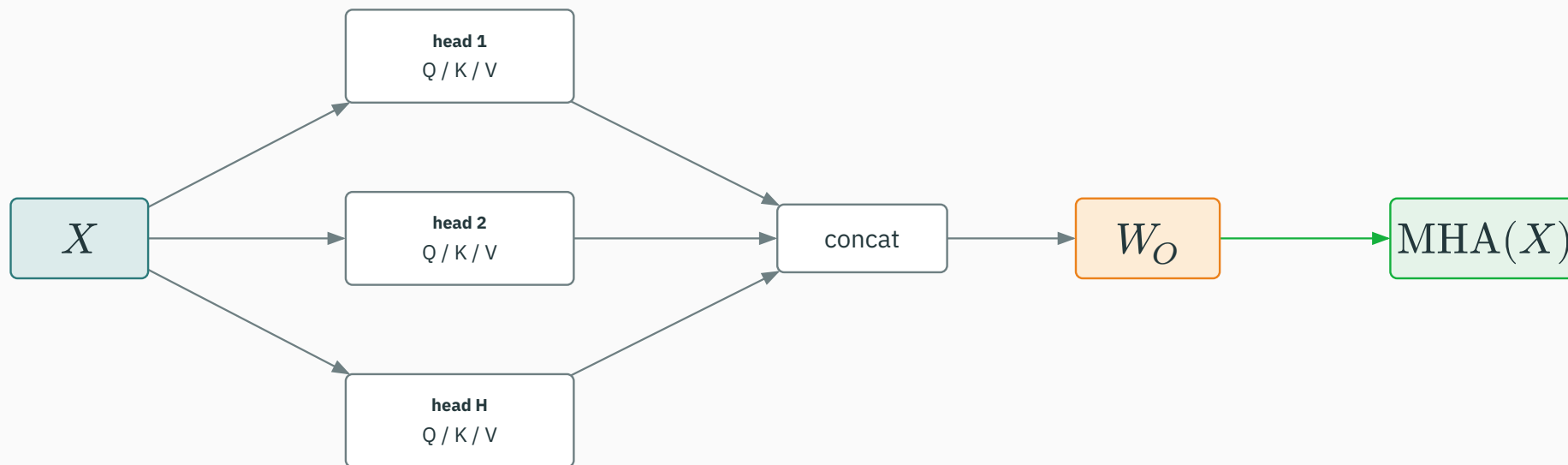
Put the head outputs side by side



$$\underline{C = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \in \mathbb{R}^{T \times H d_v}}$$

For each token, write head 1's output, then head 2's output, and so on.
Concatenation keeps all the features from every head.

Mix the concatenated features with a linear layer



$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

W_O is a learned linear layer. It combines features from the heads and produces width d_{model} , which we need for the residual addition.

Check the widths before moving on

COMMON WIDTH CHOICE

$$d_k = d_v = \frac{d_{\text{model}}}{H}$$

If $d_{\text{model}} = 512$ and $H = 8$,
then $d_k = d_v = 64$.

CONCATENATION CHECK

$$H d_v = 8 \times 64 = 512$$

so C already has width d_{model} before W_O mixes it.

Each head produces 64 features. Concatenating eight heads gives 512.

What can an attention map tell us?

INSPECT

Look at weights, outputs, gradients, and how interventions change behavior.

BE CAREFUL

A head may change roles across inputs. A large weight alone cannot prove causal importance.

TEST

Remove a component or change the input, then compare the model's behavior.

Attention weights help us inspect a model. Explaining its behavior needs further evidence.

Attention mixes rows; the FFN mixes features

ATTENTION

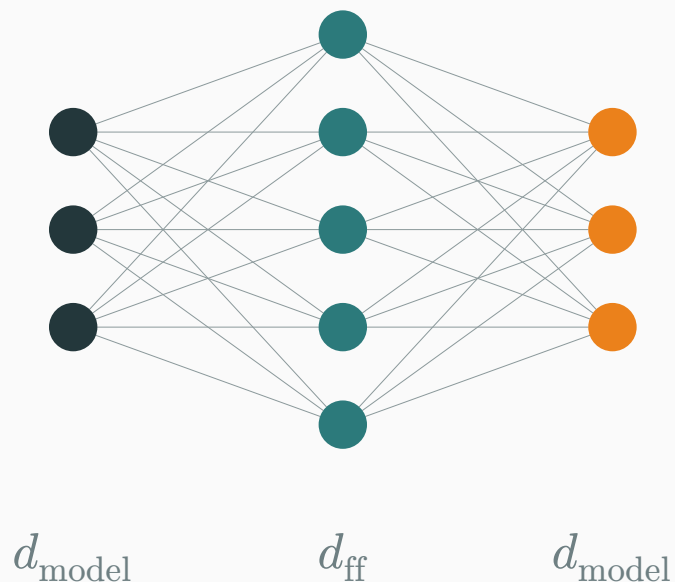
Mix information between visible token rows.

FEED-FORWARD NETWORK

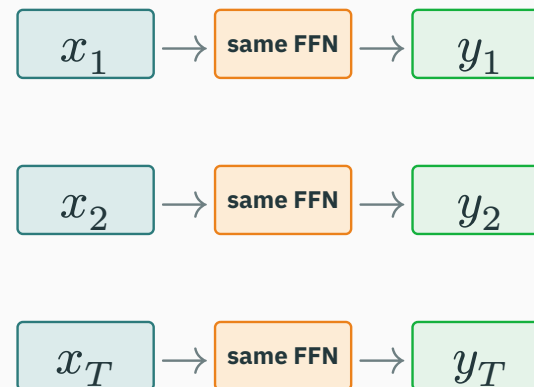
Recombine the features inside each row with a nonlinear map.

A block first mixes information across token rows, then transforms the features within each row.

Give every token the same small neural network



$$\text{FFN}(x) = W_2 \varphi(W_1 x + b_1) + b_2$$



The same MLP weights are used at every position

Across tokens, then within a token

MHA · ACROSS ROWS

	key 1	key 2	key 3
query 1	.7	.2	.1
query 2	.1	.7	.2
query 3	.2	.3	.5

token positions exchange information

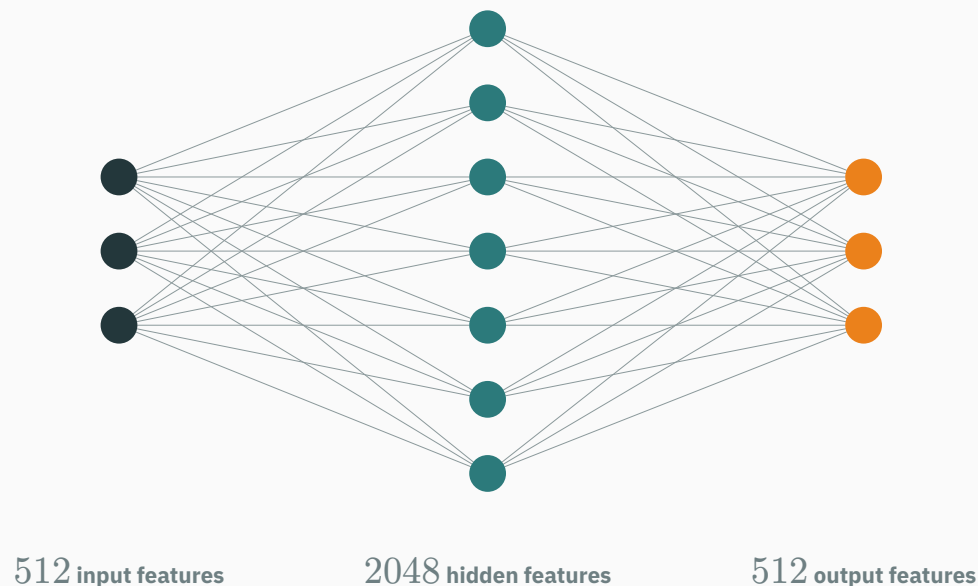
FFN · WITHIN ONE ROW



a nonlinear transformation of one row's features

Attention combines rows. The FFN transforms each row separately.

Why does the FFN widen in the middle?



$512 \rightarrow 2048 \rightarrow 512$: **compute more nonlinear features, then return to the original width.**

Can a sublayer pass its input through unchanged?

$$y = x + F(x)$$

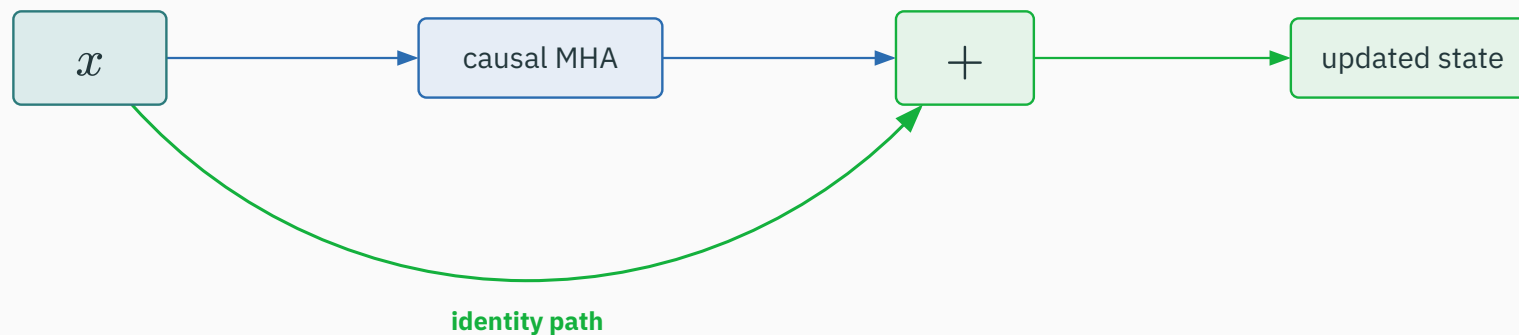
KEEP THE INPUT

If $F(x) \approx 0$, then $y \approx x$. The input passes straight through.

LEARN AN UPDATE

The sublayer learns the correction $F(x)$ to add to its input.

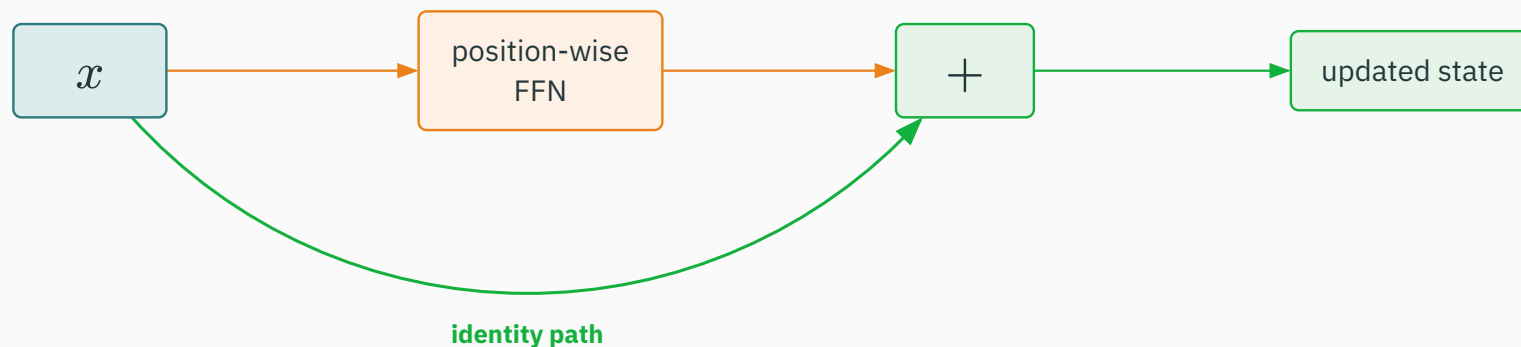
Add the attention output to its input



$$\underline{r = x + \text{MHA}(x)}$$

| Shapes must match: both x and $\text{MHA}(x)$ are $T \times d_{\text{model}}$.

Add the FFN output to its input

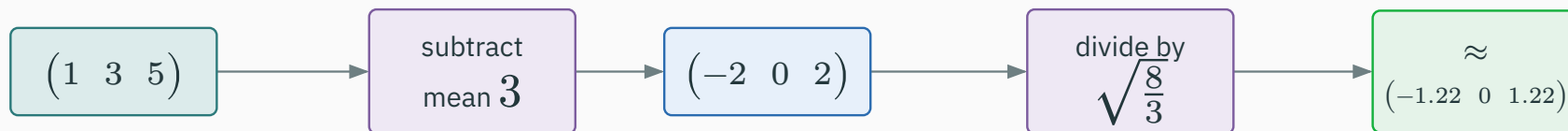


$$\underline{y = r + \text{FFN}(r)}$$

| The residual stream keeps width d_{model} through every sublayer and every block.

LayerNorm works across one token's features

Start with one token row. Subtract its mean, then divide by its standard deviation.



ONE ROW AT A TIME

GENERAL ROW x_i

$$\text{LN}(x_i) = \gamma \cdot \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta$$

Each token row has its own feature mean and variance. Learned γ, β are shared across positions.

Each token row is normalized separately

Where should LayerNorm go?

PRE-NORM

$$y = x + F(\text{LN}(x))$$

Normalize **before** the sublayer.

POST-NORM

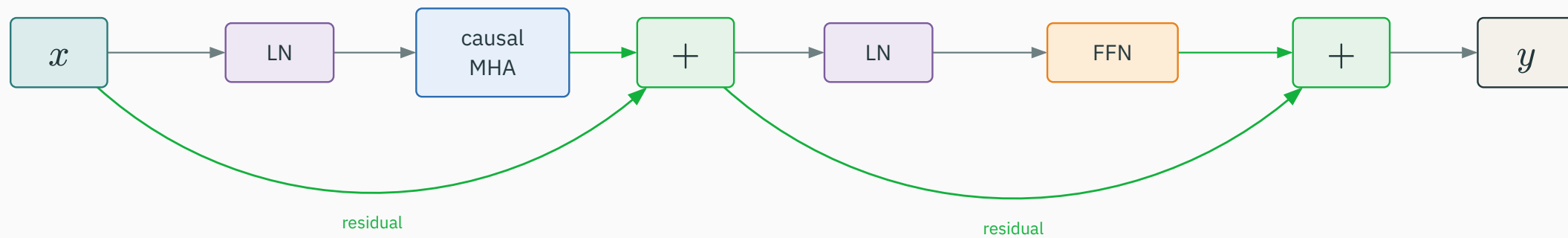
$$y = \text{LN}(x + F(x))$$

Normalize **after** residual addition.

We will use pre-norm in our block

Both versions are used. In the pre-norm drawing, we can follow the residual path around each sublayer.

Put the sublayers together



(same $T \times d_{\text{model}}$ stream)

(future positions masked)

(two residual additions)

attention → add input → FFN → add input

Write the block as two updates

ATTENTION SUBLAYER

$$u = x + \underbrace{\text{MHA}(\text{LN}(x))}_{\text{causal retrieval}}$$

FEED-FORWARD SUBLAYER

$$y = u + \underbrace{\text{FFN}(\text{LN}(u))}_{\text{per-token transformation}}$$

$x, u, y \in \mathbb{R}^{T \times d_{\text{model}}}$. The two green plus signs in the drawing are exactly the two additions above.

The block in five lines of PyTorch-style code

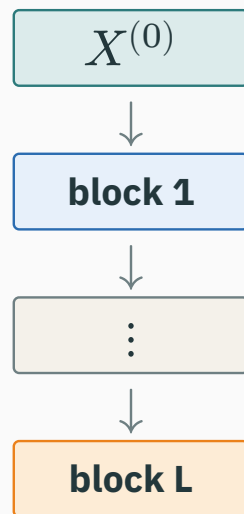
PyTorch

```
class Block(nn.Module):  
    def forward(self, x):  
        x = x + self.causal_attn(self.ln1(x))  
        x = x + self.ffn(self.ln2(x))  
        return x
```

For each sublayer: normalize its input, compute the output, and add the input back.

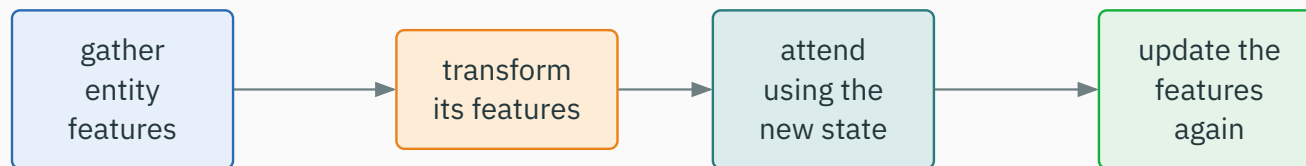
`self.causal_attn` wraps the attention calculation. We will add the full module, dropout, and initialization in the notebook.

Stack several blocks



Each block has its own attention, FFN, and LayerNorm parameters. The blocks share the same structure.

The next block uses the updated token states



FIRST BLOCK

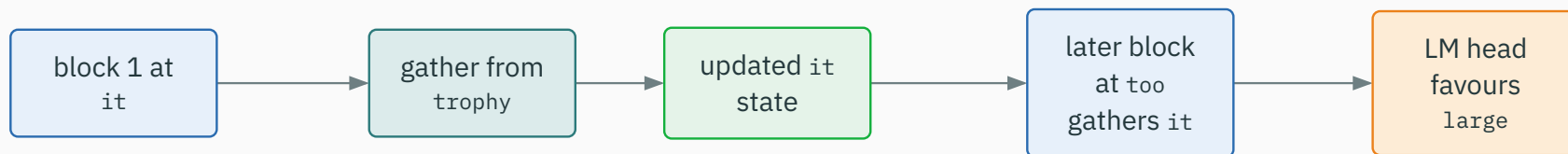
Might combine information from a name or a nearby verb in a token's state.

LATER BLOCK

Can use the updated state to compute a new set of attention weights.

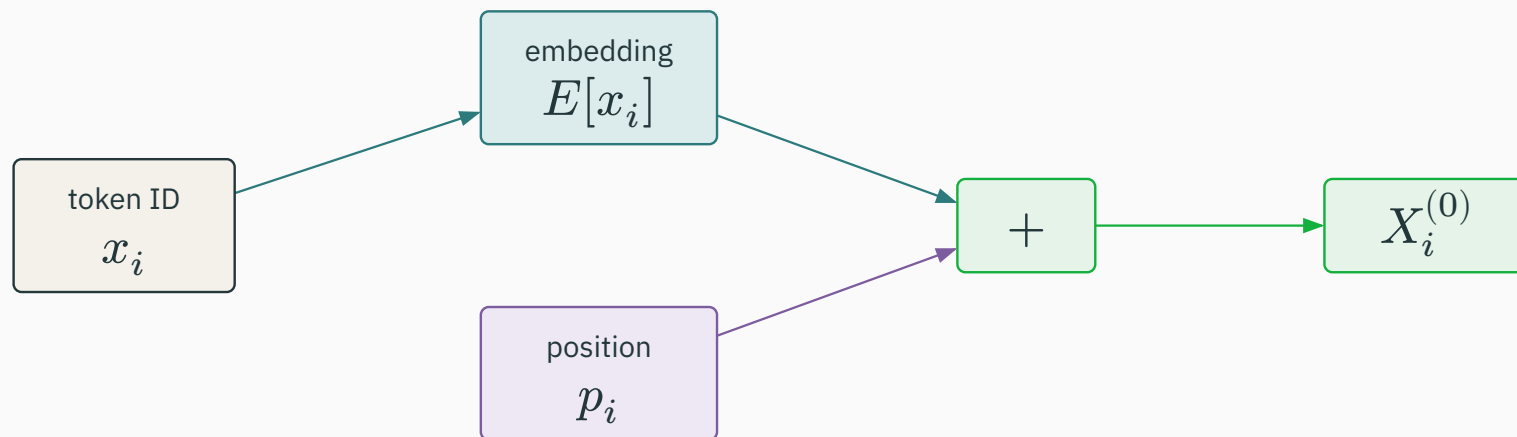
Several blocks allow several steps of computation. We cannot assume that each layer corresponds to one human reasoning step.

Information can pass through an intermediate token



The later block uses the updated state at it. This picture illustrates one possible computation.

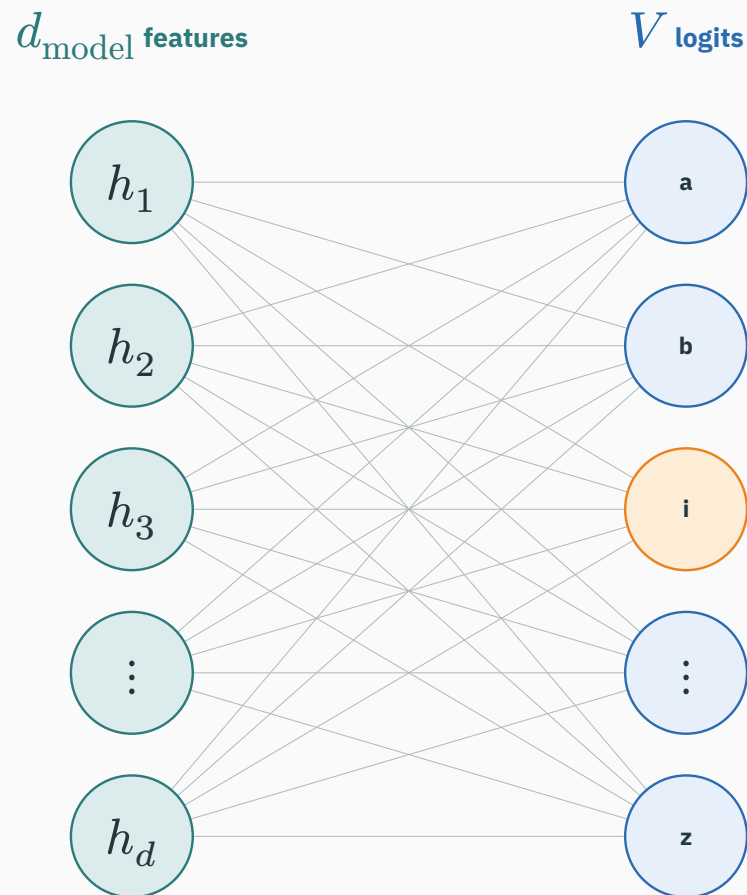
Begin with token and position information



$$X_i^{(0)} = E[x_i] + p_i$$

Without p_i , identical tokens begin with identical rows. Position information tells the model where each copy occurs. Some modern models add position information inside attention instead.

Turn the final token state into token scores



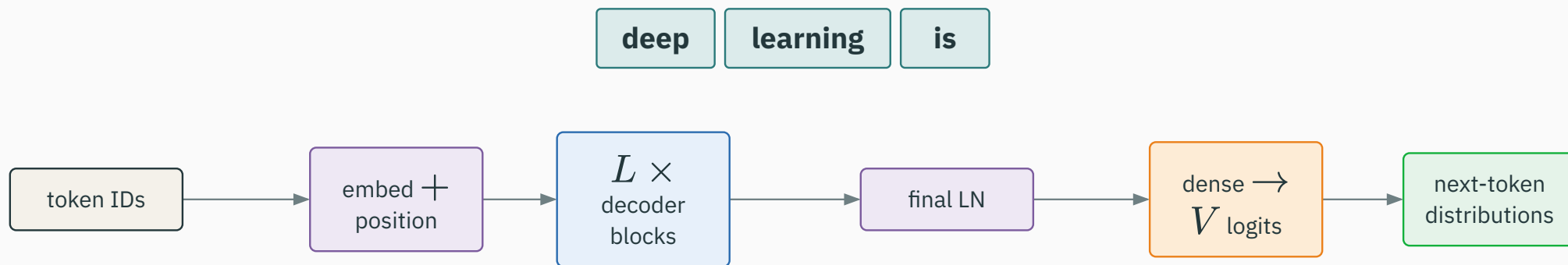
$$z_t = h_t W_{\text{vocab}} + b$$

$$p_t = \text{softmax}(z_t)$$

$$\begin{matrix} (h_t : d_{\text{model}}) \\ (z_t : V) \end{matrix}$$

softmax runs over the V vocabulary scores

The decoder-only model from end to end



The output is still a probability distribution over the next token.

Slide the targets one place to the left

	POSITION 1	POSITION 2	POSITION 3	POSITION 4
input	<BOS>	deep	learning	is
target	deep	learning	is	fun

Each row uses its prefix to predict the next token

During training, we know the whole sentence

DURING ONE TRAINING PASS

Feed in the true sentence. Row t can use the true prefix $x_{\leq t}$.

The causal mask hides future tokens from each row, so a row cannot see the answer it must predict.

ALL ROWS TOGETHER

Score the next token at every row in the same pass. Sampling and feeding a token back happens during generation.

Score every next-token guess

$$\mathcal{L} = - \sum_{t=1}^{T-1} m_t \log p_{\theta}(x_{t+1} \mid x_{\leq t})$$

LOOK ONLY LEFT

The causal mask keeps future words hidden.

SCORE REAL ROWS

m_t skips padding or ignored positions.

SCORE THE TARGET

Cross-entropy uses the probability of the token one step to the right.

Code often averages over valid tokens. The sum above shows one loss term for each next-token prediction.

The next-token objective stays the same

FIXED-WINDOW CHARACTER MODEL

context: a a b

target: i

loss: $-\log p_{\theta}(i \mid a, a, b)$

DECODER TRANSFORMER

context: $x_{\leq t}$

target: x_{t+1}

loss: $-\log p_{\theta}(x_{t+1} \mid x_{\leq t})$

Both models minimize the loss on the next token

Shift the targets and compute cross-entropy

PyTorch

```
inputs  = tokens[:, :-1]
targets = tokens[:, 1:]
logits  = model(inputs)
loss = F.cross_entropy(
    logits.flatten(0, 1), targets.flatten())
```

The two slices pair each input position with its next-token target.

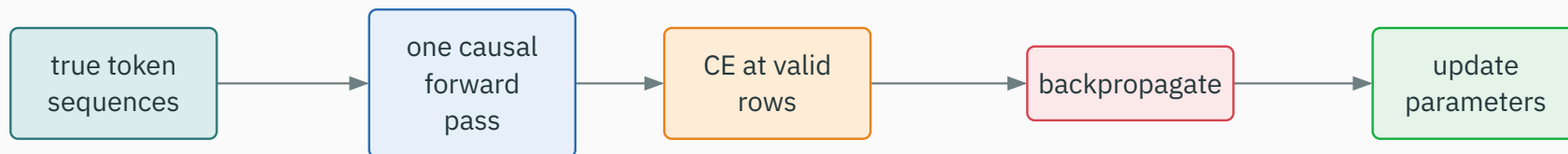
For padded batches, pass the padding token ID as `ignore_index`. We will add the data loader and masks in the notebook.

Training: one pass, many next-token guesses

TRAIN tokens $\rightarrow$ logits $\rightarrow$ loss $\rightarrow$ update

GENERATE prefix $\rightarrow$ distribution $\rightarrow$ choose
 $\rightarrow$ append

EVALUATE held-out tokens $\rightarrow$ mean NLL / PPL



Positions are processed in parallel during the training forward pass. Layers are still evaluated one after another.

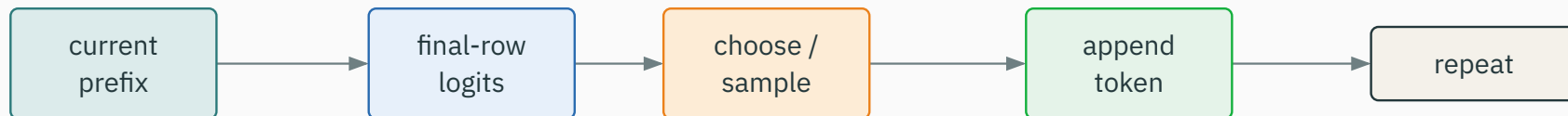
Generation: one new token, then go again

TRAIN tokens $\rightarrow$ logits $\rightarrow$ loss $\rightarrow$ update

GENERATE prefix $\rightarrow$ distribution $\rightarrow$ choose

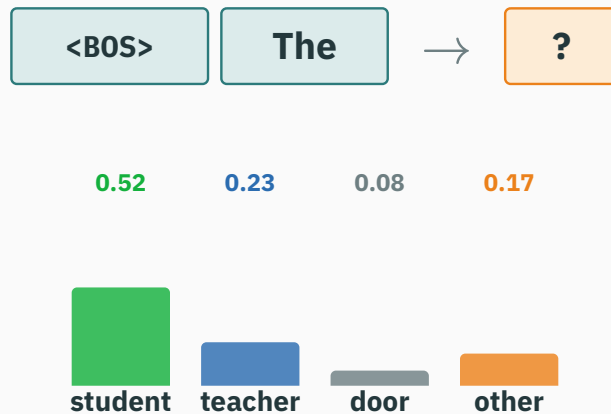
EVALUATE held-out tokens $\rightarrow$ mean NLL / PPL

$\rightarrow$ append



Autoregressive generation is sequential because the next model input depends on the token just chosen.

Generation, first step



choose student → append it to the prefix

| Use the distribution at the final input position to choose the next token.

Generation, one step later



0.47 0.25 0.16 0.12



choose opened → continue with a longer prefix

The prefix now has one more token. We can cache calculations from the earlier positions and reuse them.

The whole generation loop

PyTorch

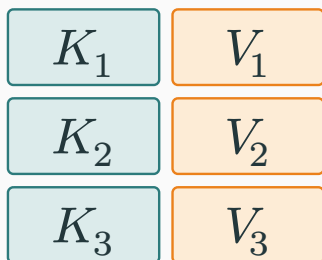
```
for _ in range(max_new_tokens):  
    logits = model(ids)[: , -1]  
    probs = logits.softmax(dim=-1)  
    next_id = torch.multinomial(probs, 1)  
    ids = torch.cat([ids, next_id], dim=1)
```

Score the prefix, sample one token, append it, and repeat.

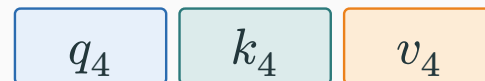
Greedy decoding would use `argmax` instead of sampling. A real implementation also handles stopping tokens and the KV cache.

Cache the keys and values from earlier positions

ALREADY CACHED AT EACH LAYER



NEW TOKEN



Score q_4 against cached $K_{1:3}$ and new k_4 ; append k_4, v_4 to the cache.

In causal incremental decoding, adding a future token leaves earlier hidden rows unchanged. Their projected keys and values stay the same too.

What does a longer context cost?

TRAINING SCORE GRID

	k_1	k_2	k_3	k_4
q_1	•	×	×	×
q_2	•	•	×	×
q_3	•	•	•	×
q_4	•	•	•	•

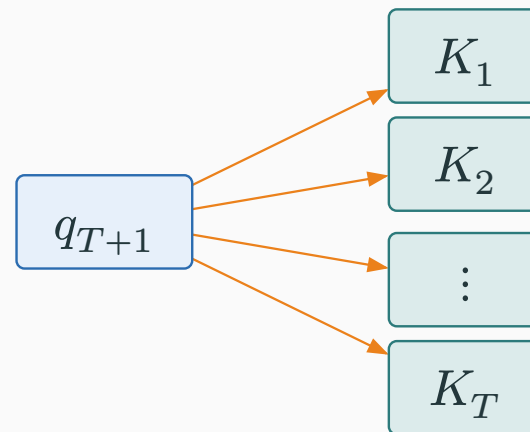
$\Theta(T^2)$ pair scores

DECODING CACHE

K_1	V_1
K_2	V_2
$\vdots$	$\vdots$
K_T	V_T

stored rows grow with T

ONE NEW QUERY

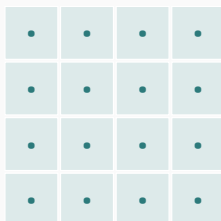


compare with all cached keys

Optimized kernels can avoid materializing the full $T \times T$ matrix, but they do not remove the vanilla all-pairs arithmetic.

Three common ways to arrange Transformer blocks

encoder-only



VISIBILITY

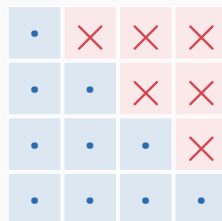
all tokens can see both directions

COMMON OBJECTIVE

masked-token / representation learning

Example: BERT

decoder-only



VISIBILITY

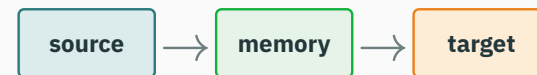
each token sees only its prefix

COMMON OBJECTIVE

next-token prediction

Example: GPT-style LM

encoder-decoder



VISIBILITY

source bidirectional; target causal

COMMON OBJECTIVE

conditioned sequence generation

Example: translation

Encoder-only: read the whole input

keys →	the	river	bank	was	wide
bank →	.12	.39	.18	.10	.21

VISIBILITY

Every position may attend left and right; there is no causal triangle.

TYPICAL USE

Build contextual features for classification, tagging, retrieval, or masked-token training.

| BERT uses an encoder. The next-token model we built uses a causal decoder.

Decoder-only: read only the prefix

keys →	<BOS>	The	cat	sat	on
sat →	.08	.14	.43	.35	0

VISIBILITY

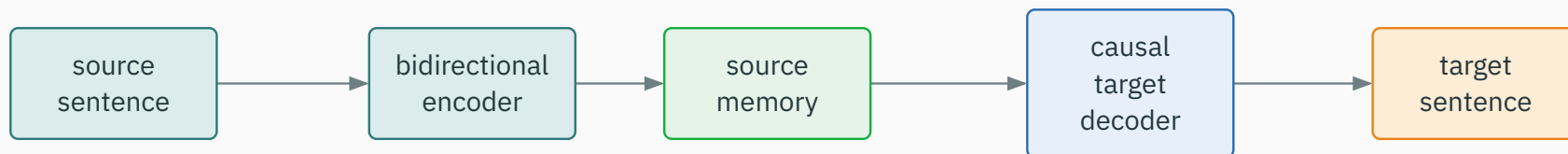
Position t attends only to positions $\leq t$.

OBJECTIVE

Predict x_{t+1} from $x_{\leq t}$ at every valid row.

This is the decoder we have been building.

Encoder-decoder: read one sequence, write another



The decoder uses causal self-attention over the generated target **and** cross-attention to the encoded source. Translation is the classic example.

Self-attention and cross-attention use the same calculation

SELF-ATTENTION

$$Q = XW_Q$$

$$K = XW_K \quad V = XW_V$$

Queries, keys, and values come from the same sequence.

CROSS-ATTENTION

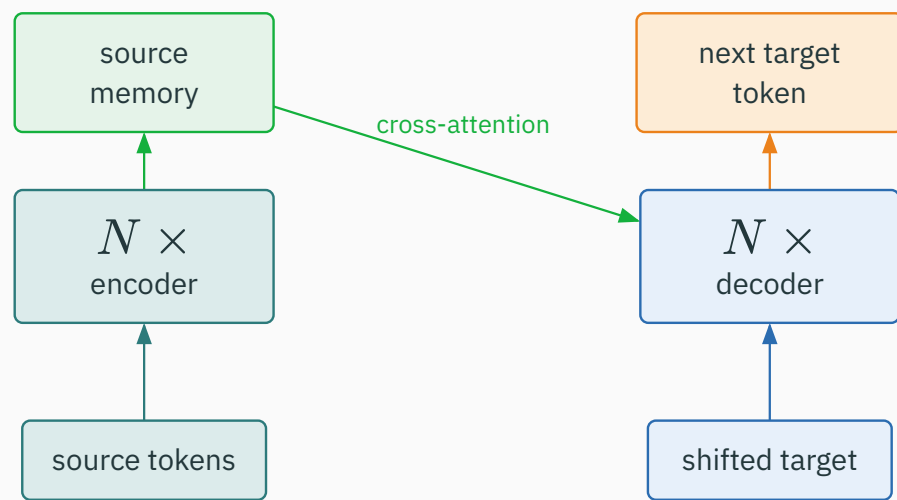
$$Q = Y_{\text{decoder}}W_Q$$

$$K = X_{\text{source}}W_K \quad V = X_{\text{source}}W_V$$

Queries come from the decoder. Keys and values come from the encoded source.

The attention calculation stays the same. What changes is the source of the queries, keys, and values.

The 2017 Transformer had an encoder and a decoder



2017

Vaswani et al., **Attention Is All You Need**, introduced the Transformer as an encoder-decoder model for sequence transduction.

We focused on a decoder-only model because it directly extends next-token prediction.

Vaswani et al. (2017) · arXiv:1706.03762

Put the sequence models side by side

model	available context	path to far token	training across time	main trade-off
fixed-window MLP	exactly k previous tokens	1 within window; unavailable outside	parallel examples	input width and parameters tied to k
RNN	prefix compressed into recurrent state	$O(T)$ recurrent steps	sequential recurrence	compact state; long dependency path
temporal convolution	finite receptive field grows with depth/dilation	depends on kernel and depth	parallel	local bias; depth needed for reach
self-attention	all visible tokens in the mask	1 attention edge per layer	parallel masked rows	vanilla all-pairs score cost

The path length counts the steps information must take through the model. A short path does not guarantee that the model learns the dependency.

Longer memory has a price

FIXED MLP

Uses a fixed window k . A larger context needs a different first layer.

RNN

Roughly linear steps in T , but those recurrent steps are sequential.

SELF-ATTENTION

Vanilla pair scores are $\Theta(T^2)$; a causal triangle is still $\Theta(T^2)$.

Compare both what context a model can use and what that access costs.

These complexity terms omit width, layers, constants, kernels, and hardware.
Each architecture has trade-offs.

What does attention give us?

GIVES

CONTENT-DEPENDENT ROUTING

A query can weight visible sources differently for each token and layer.

SHORT COMMUNICATION PATHS

Any visible token can directly contribute within an attention sublayer.

DOES NOT GUARANTEE

TRUTH OR REASONING

The model may still generate false or poorly justified text.

EXPLANATION OR UNLIMITED MEMORY

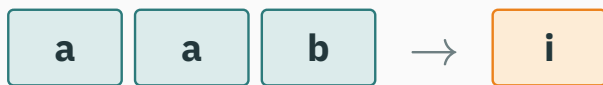
Attention weights are not explanations. Context length and the KV cache remain finite.

Four quick checks

CLAIM	CORRECTION
“Attention weights are explanations.”	We can inspect the weights, but an explanation needs stronger causal evidence.
“One head learns one linguistic rule.”	A pattern may be spread across heads and may change across inputs.
“Masking is only used at generation.”	We also need the causal mask during parallel next-token training.
“A Transformer removes the MLP.”	Every standard block contains a substantial position-wise FFN.

Compare our two language models

FIXED-WINDOW MLP



DECODER TRANSFORMER



Both use IDs → embeddings → logits → cross-entropy → sampling

The MLP concatenates three embedding rows. The Transformer keeps one row per token and combines information between rows through causal attention.

Can you finish this sentence?

“A Transformer language model differs from our character MLP mainly because ...”

...each position can weight earlier tokens differently, combine their values, pass the result through an MLP, and repeat this in the next block.

We still train it to predict the next token from the previous tokens.

References and lecture plan

References

NEXT-TOKEN PREDICTION

Nipun Batra · Next Token Prediction

aabid, embeddings, concatenation, classification, sampling

LANGUAGE MODELS

Stanford CME 295 · Lecture 1: Transformer

language tasks, tokenization, and the full Transformer

SEQUENCE MODELS

MIT 6.S191 · Deep Sequence Modeling

queries, keys, values, and self-attention

THE ORIGINAL TRANSFORMER

Vaswani et al. · Attention Is All You Need

scaled dot-product attention and the encoder-decoder Transformer

Technical diagrams were drawn in Typst/Fletcher. Illustrations were generated with OpenAI image generation; labels were added in Typst. Schematic token splits and attention maps are labeled as teaching examples.

The three lectures

- Lecture 1** Text → tokens → next-token prediction
- Lecture 2** Context → weighted averages → attention
- Lecture 3** Multi-head attention + MLP + residual paths → Transformer

SLIDES

Use the presentation PDF to reveal each step. The handout shows completed slides.

LECTURE PLAN

The Markdown outline records examples, questions, and suggested stopping points.